Trường Đại học Khoa học Tự nhiên TP.HCM
Khoa Điện tử - Viễn thông

---☆---

BÁO CÁO ĐỒ ÁN CUỐI KHÓA

ĐỀ TÀI: NHẬN DIỆN MŨ BẢO HIỂM VÀ ĐIỀU KHIỂN ĐÈN GIAO THÔNG ỨNG DỤNG HỌC SÂU TRÊN VI ĐIỀU KHIỂN ESP32-S3 (MOTORCYCLE HELMET IDENTIFICATION AND TRAFFIC LIGHT CONTROL)

Môn học: Ứng dụng Học sâu và Thị giác máy tính trong Điện tử - Viễn thông

Thành viên nhóm		
STT	Họ và tên	MSSV
1	Vương Đức Phát	25300066
2	Phùng Gia Tân	25300070
3	Đỗ Xuân Hùng	25300064

Trường Đại học Khoa học Tự nhiên TP.HCM
Khoa Điện tử - Viễn thông

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

TRUNG TÂM TIN HỌC

BÁO CÁO ĐỒ ÁN CUỐI KHÓA

Môn học

Ứng dụng Học sâu và Thị giác máy tính trong Điện tử - Viễn thông

NHẬN DIỆN MŨ BẢO HIỂM VÀ ĐIỀU KHIỂN ĐÈN GIAO THÔNG ỨNG DỤNG HỌC SÂU TRÊN VI ĐIỀU KHIỂN ESP32-S3 (MOTORCYCLE HELMET IDENTIFICATION AND TRAFFIC LIGHT CONTROL)

Học viên thực hiện: Vương Đức Phát – 25300066

Phùng Gia Tân – 25300070

Đỗ Xuân Hùng - 25300064

Lớp/khóa: 25ICD1/ DTVT-Deep Learning and Computer Vision 2026

Giảng viên hướng dẫn: Hồ Nhựt Minh

Thời gian thực hiện: Từ tháng 09/2026 đến tháng 10/2026

Liên kết youtube: https://www.youtube.com/shorts/QQKaPLMMSUQ

Thành phố Hồ Chí Minh – Tháng 10/2026

THÔNG TIN VÀ PHÂN CÔNG ĐỒ ÁN

Bảng 1. Quy cách trình bày áp dụng trong báo cáo

Thành phần	Quy cách áp dụng
Khổ giấy và lề trang	A4 (21,0 × 29,7 cm); lề trái 2,5 cm; lề trên, dưới, phải 2,0 cm
Phông chữ và giãn dòng	Times New Roman 13 pt, giãn dòng 1,5; bảng biểu và chú thích 10–11 pt
Mã nguồn đính kèm	Consolas 9,5 pt, đặt trong khung nền xám nhạt
Đánh số bảng và hình	Bảng: tên đặt phía trên; Hình: tên đặt phía dưới; dẫn chiếu trực tiếp trong bài
Trích dẫn tài liệu	Dạng [số] theo thứ tự xuất hiện; đầy đủ tác giả, tiêu đề, URL và ngày truy cập

Bảng 2. Thông tin chung của đồ án

Nội dung	Thông tin chi tiết
Tên đề tài tiếng Việt	Nhận diện mũ bảo hiểm và điều khiển đèn giao thông ứng dụng Học sâu trên ESP32-S3
Tên đề tài tiếng Anh	Motorcycle Helmet Identification and Traffic Light Control using Deep Learning and ESP32-S3
Loại bài toán	Phân loại ảnh nhị phân (Image Classification) kết hợp điều khiển ngoại vi GPIO
Mục tiêu ứng dụng	Phát hiện tình trạng đội mũ bảo hiểm (helmet / without_helmet) từ luồng camera và tự động chuyển trạng thái đèn tín hiệu Đỏ
Thiết bị triển khai	Bo mạch MKE-K01 ESP32-S3 N16 (16 MB Flash, không PSRAM), kết nối Webcam Laptop qua giao tiếp USB-UART ở tốc độ 460.800 baud
Dự án Edge Impulse của nhóm	Owner: gtanoc | Project name: gtanoc-project-1 | Project ID: 1128769 | Deploy Version: 18 (Library v1.0.18)
Repository / Mã nguồn	[https://github.com/giatan0601/helmet-detection] [https://github.com/PhatVuong1208/Helmet-detection-using-ESP32S2]
Ngày chốt dữ liệu & mô hình	Tháng 10/2026 (Biên dịch thư viện gtanoc-project-1_inferencing v1.0.18)

Bảng 3. Phân công công việc và mức độ đóng góp của các thành viên

Thành viên	Hạng mục phụ trách chính	Sản phẩm và bằng chứng bàn giao	Tỷ lệ
Thành viên 1: [[Phùng Gia Tân – 25300070]	Xây dựng tập dữ liệu, tiền xử lý ảnh, thiết kế Impulse, huấn luyện mô hình phân loại ảnh và tối ưu hóa EON Compiler trên Edge Impulse Studio.	Project ID 1128769, đồ thị hội tụ Loss, bảng đánh giá Validation/Test và gói thư viện gtanoc-project-1_inferencing (v1.0.18).	33,3333333%
Thành viên 2: [Đỗ Xuân Hùng - 25300054]	Lập trình firmware nhận khung hình RGB thô trên ESP32-S3, quản lý bộ đệm SRAM nội, đóng gói pixel 24-bit và gọi SDK suy luận.	Mã nguồn cơ sở trên Arduino IDE, cơ chế bắt tay READY:W:H:C, hàm chuyển đổi raw_feature_get_data() và đo bộ nhớ Free Heap.	33,3333333%
Thành viên 3: [Vương Đức Phát - 25300066]	Thiết kế mạch đèn giao thông trên breadboard, viết thuật toán điều khiển LED, đồng bộ kịch bản Python truyền ảnh tốc độ 460.800 baud, kiểm thử thực tế, quay video và biên soạn báo cáo.	Mạch phần cứng GPIO 4/GPIO 5, tệp hoàn chỉnh helmet_detect.ino và webcam.py, nhật ký thử nghiệm thực tế, video demo và báo cáo tổng kết.	33,3333333%

CAM KẾT VÀ GHI NHẬN NGUỒN

Nhóm thực hiện cam kết các số liệu cấu hình, mã nguồn điều khiển và nhật ký thực nghiệm trình bày trong báo cáo này là kết quả làm việc thực tế của nhóm trên dự án Edge Impulse ID 1128769 và bo mạch MKE-K01 ESP32-S3 N16. Các thư viện mã nguồn mở, công cụ phát triển và tài liệu hướng dẫn bên ngoài đều được trích dẫn rõ ràng tại mục Tài liệu tham khảo.

Bảng 4. Đối chiếu phần kế thừa và phần nhóm tự phát triển

Thành phần	Nguồn và giấy phép	Nội dung nhóm điều chỉnh / phát triển mới	Bằng chứng trong gói nộp

Bộ dữ liệu ảnh mũ bảo hiểm	[https://universe.roboflow.com/tanuja-vinchurkar/helmet-detection-kkt4z-409pq/browse?queryText=&pageSize=100&startingIndex=0&browseQuery=true]	Chọn lọc ảnh, gán nhãn 2 lớp (helmet, without_helmet), chia tập Train/Validation/Test và chuẩn hóa về kích thước 64×64 RGB.	Project ID 1128769 trên Edge Impulse Studio
SDK suy luận biên	Edge Impulse C++ SDK & TensorFlow Lite)	Biên dịch mô hình lượng tử hóa INT8 bằng EON Compiler (RAM optimized) giảm Tensor Arena còn 90.419 bytes (v1.0.18).	Thư mục src/ và tệp model_info.json
Firmware ESP32-S3 (helmet_detect.ino)	Tự phát triển dựa trên tài liệu API của Edge Impulse và Arduino-ESP32	Bỏ phụ thuộc PSRAM và esp_camera; cấp phát tĩnh rgb888_buf[12288] trên SRAM; nhận luồng ảnh qua UART 460.800 baud và điều khiển LED tại GPIO 4, GPIO 5.	src/helmet_detect.ino
Phần mềm truyền ảnh (webcam.py)	Tự phát triển bằng Python (OpenCV + PySerial)	Tự động đọc cấu hình READY:W:H:C từ mạch, cắt khung vuông tâm ảnh, resize INTER_AREA và chia gói 256 bytes chống tràn bộ đệm.	src/webcam.py

LỜI CẢM ƠN

Nhóm xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn [... — Điền học vị và họ tên giảng viên] cùng các thầy cô tại Trung tâm Tin học – Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM đã truyền đạt kiến thức nền tảng và kinh nghiệm thực hành quý báu trong suốt học phần Ứng dụng Học sâu và Thị giác máy tính trong Điện tử - Viễn thông. Những góp ý sát sao về cách kiểm soát bộ nhớ vi điều khiển, thiết kế đường truyền dữ liệu và phương pháp đánh giá mô hình độc lập đã giúp nhóm tháo gỡ nhiều vướng mắc kỹ thuật để hoàn thành đồ án này.

TÓM TẮT

Đồ án xây dựng hệ thống nhận diện việc chấp hành đội mũ bảo hiểm và điều khiển đèn tín hiệu giao thông mô phỏng tại các chốt kiểm soát hoặc cổng ra vào khu nội bộ. Do bo mạch triển khai thực tế của nhóm là MKE-K01 ESP32-S3 N16 (16 MB Flash, không tích hợp PSRAM ngoài và không gắn sẵn cảm biến camera trên mạch), nhóm đã thiết kế kiến trúc phối hợp giữa máy tính thu hình (Host PC) và vi điều khiển suy luận tại biên (Edge MCU). Máy tính thu hình từ Webcam, cắt vùng vuông trung tâm, thu nhỏ về kích thước chuẩn và truyền dữ liệu điểm ảnh thô xuống ESP32-S3 qua giao tiếp USB-UART ở tốc độ 460.800 baud. Trên nền tảng Edge Impulse Studio (Project ID 1128769, phiên bản triển khai 18), mô hình phân loại ảnh nhị phân (helmet và without_helmet) với đầu vào 64×64 RGB được lượng tử hóa sang số nguyên 8-bit (INT8) và tối ưu hóa bằng EON Compiler. Kết quả đánh giá mô hình đạt F1-score 84,5% trên tập Validation (Precision 0,83; Recall 0,86) và độ chính xác 89,29% trên tập Kiểm thử (Model testing, F1-score đạt 0,93). Khi nhúng xuống ESP32-S3, vùng nhớ Tensor Arena chỉ chiếm 90.419 bytes kết hợp với bộ đệm ảnh tĩnh 12.288 bytes, cho phép hệ thống vận hành ổn định hoàn toàn trên SRAM nội (Free Heap còn trống khoảng 359 KB) và điều khiển tức thời đèn LED Đỏ (GPIO 4) và LED Xanh (GPIO 5).

Nhận diện mũ bảo hiểm, ESP32-S3 không PSRAM, Edge Impulse, EON Compiler, Phân loại ảnh INT8, Điều khiển đèn giao thông.Từ khóa:

ABSTRACT

Motorcycle Helmet Identification and Traffic Light Control using Deep Learning and ESP32-S3. Project Title:

This project develops an automated helmet compliance identification and traffic light control system tailored for access gates and simulated traffic checkpoints. To overcome the hardware constraints of the MKE-K01 ESP32-S3 N16 development board (16 MB Flash, without external PSRAM or an onboard camera module), our team implemented a cooperative Host-PC to Edge-MCU architecture. A Python script on the laptop captures live video frames, crops the central square region, resizes the image, and streams raw RGB

pixel bytes over a 460,800-baud USB-UART link in 256-byte chunks. On Edge Impulse Studio (Project ID 1128769, deploy version 18), an image classification model distinguishing between 'helmet' and 'without_helmet' (64×64 RGB input) was quantized to INT8 and compiled with the RAM-optimized EON Compiler. The model achieved an 84.5% F1-score on the validation set and 89.29% accuracy (0.93 F1-score) on the testing set. On the ESP32-S3, the model requires a Tensor Arena of 90,419 bytes alongside a static 12,288-byte frame buffer, running reliably within internal SRAM (~359 KB free heap) to trigger Red (GPIO 4) and Green (GPIO 5) traffic indicator LEDs.

Helmet Identification, ESP32-S3 SRAM, Edge Impulse, EON Compiler, INT8 Image Classification, Traffic Light Control. Keywords:

DANH MỤC CHỮ VIẾT TẮT VÀ KÝ HIỆU

Bảng 5. Danh mục từ viết tắt sử dụng trong báo cáo

Ký hiệu	Tên đầy đủ tiếng Anh	Ý nghĩa sử dụng trong đồ án
AI / DL	Artificial Intelligence / Deep Learning	Trí tuệ nhân tạo / Học sâu
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập xử lý dữ liệu ảnh 2 chiều
CV	Computer Vision	Thị giác máy tính
DSP	Digital Signal Processing	Khối xử lý tín hiệu số / tiền xử lý ảnh đầu vào
EON	Edge Optimized Neural (Compiler)	Trình biên dịch mạng nơ-ron tối ưu bộ nhớ của Edge Impulse
FOMO	Faster Objects, More Objects	Thuật toán phát hiện vật thể theo tâm (đối chứng trong quá trình tối ưu)
GPIO	General Purpose Input/Output	Chân vào/ra đa năng trên vi điều khiển dùng kích hoạt đèn LED
INT8 / FP32	8-bit Integer / 32-bit Floating Point	Kiểu dữ liệu số nguyên 8-bit lượng tử hóa và số thực 32-bit
MCU	Microcontroller Unit	Vi điều khiển trung tâm (ESP32-S3)
PSRAM / SRAM	Pseudo Static RAM / Static RAM	Bộ nhớ RAM mở rộng bên ngoài và bộ nhớ RAM tĩnh nội bộ trong chip

RGB888	Red - Green - Blue (8-bit per channel)	Định dạng ảnh màu 3 kênh, 24-bit mỗi điểm ảnh (3 bytes/pixel)
TFLM	TensorFlow Lite for Microcontrollers	Thư viện suy luận mô hình học sâu trên vi điều khiển
UART	Universal Asynchronous Receiver-Transmitter	Giao thức truyền thông nối tiếp không đồng bộ qua cáp USB

MỤC LỤC

CAM KẾT VÀ GHI NHẬN NGUỒN 4

LỜI CẢM ƠN 5

TÓM TẮT 6

ABSTRACT 6

DANH MỤC CHỮ VIẾT TẮT VÀ KÝ HIỆU 7

1.1 Bối cảnh và vấn đề cần giải quyết 15

1.2 Mục tiêu và tiêu chí nghiệm thu 15

1.3 Phạm vi, giả định và phần không thực hiện 16

1.4 Dự án tham chiếu và đóng góp của nhóm 17

1.5 Kế hoạch thực hiện và cấu trúc báo cáo 17

Chương 2. CƠ SỞ LÝ THUYẾT 19

2.1 Bài toán học có giám sát và vòng đời mô hình 19

2.2 Biểu diễn ảnh số và dung lượng bộ nhớ khung hình 19

2.3 So sánh bài toán Phân loại ảnh (Image Classification) và Phát hiện tâm vật thể (FOMO) 20

2.4 Mạng nơ-ron tích chập (CNN) và hàm mất mát 20

2.5 Các chỉ số đánh giá chất lượng mô hình 21

2.6 Suy luận biên (Edge Inference), Lượng tử hóa INT8 và EON Compiler 21

Chương 3. THIẾT KẾ HỆ THỐNG VÀ XÂY DỰNG DỮ LIỆU 22

3.1 Kiến trúc tổng thể và luồng dữ liệu 22

3.2 Nguồn dữ liệu, quyền sử dụng và kế hoạch thu thập 22

3.3 Định dạng, kiểm soát chất lượng và phân bố đặc trưng (Feature Explorer) 23

3.4 Chia tập Train, Validation và Test 24

3.5 Kịch bản đầu ra và cơ chế xử lý lỗi truyền thông 25
Chương 4. TIỀN XỬ LÝ ẢNH VÀ ĐỒNG BỘ DỮ LIỆU GIỮA PC VÀ FIRMWARE 26
4.1 Hợp đồng dữ liệu (Data Contract) giữa Edge Impulse, Python và ESP32-S3 26
4.2 Quy trình tiền xử lý ảnh trên máy tính (webcam.py) 26
4.3 Cơ chế giải mã và đóng gói Pixel 24-bit trên ESP32-S3 27
Chương 5. HUẤN LUYỆN, SO SÁNH VÀ TỐI ƯU MÔ HÌNH 29
5.1 Thiết kế thí nghiệm và quá trình tiến hóa mô hình 29
5.2 Cấu hình huấn luyện của mô hình hoàn thiện (Version 18) 29
5.3 Phân tích đường cong học (Learning Curves) 30
5.4 Kết quả đánh giá trên tập Validation và ước lượng tài nguyên 31
5.5 Lựa chọn ngưỡng quyết định (Confidence Threshold) cho ứng dụng 31
Chương 6. TRIỂN KHAI TRÊN PHẦN CỨNG ESP32-S3 33
6.1 Danh mục linh kiện (BOM) và cấu hình biên dịch 33
6.2 Sơ đồ đấu nối phần cứng 34
6.3 Ngân sách bộ nhớ (Memory Budget) và Thời gian xử lý (Timing Budget) 35
Chương 7. THỰC NGHIỆM, KẾT QUẢ VÀ THẢO LUẬN 37
7.1 Kết quả đánh giá mô hình trên tập Kiểm thử độc lập (Model Testing) 37
7.2 Kết quả khởi chạy trên bo mạch ESP32-S3 và nhật ký kiểm thử thực tế 38
7.3 Phân tích các sự cố kỹ thuật đã xử lý trong quá trình phát triển 39
Chương 8. KẾT LUẬN, HƯỚNG PHÁT TRIỂN VÀ BÀN GIAO 41
8.1 Kết luận đối chiếu với mục tiêu đề ra 41
8.2 Hạn chế hiện tại và hướng phát triển tiếp theo 41
8.3 Danh mục sản phẩm bàn giao và hướng dẫn chạy lại nhanh 42
TÀI LIỆU THAM KHẢO 44
PHỤ LỤC A. MÃ NGUỒN HOÀN CHỈNH CỦA HỆ THỐNG (FINAL VERSION) 45

PHỤ LỤC B. DANH SÁCH KIỂM TRA (CHECKLIST) TRƯỚC KHI NỘP BÀI.....50

PHỤ LỤC C. CÁCH ĐỌC DỰ ÁN THAM CHIẾU 920883 52

PHỤ LỤC D. THAO TÁC EDGE IMPULSE VÀ BẰNG CHỨNG CẦN LƯU 54

PHỤ LỤC E. CẤU TRÚC GÓI DỰ ÁN VÀ CÁCH CHẠY LẠI 58

PHỤ LỤC F. PHIẾU THỰC NGHIỆM VÀ LOG 60

PHỤ LỤC G. CHECKLIST HOÀN THIỆN VÀ CHUẨN BỊ DEMO 61

PHỤ LỤC H. MẪU GIẢI THÍCH HÌNH VÀ BẢNG 63

DANH MỤC HÌNH

Không tìm thấy mục nhập nào của bảng hình minh họa.

DANH MỤC BẢNG

Bảng 1. Quy cách mặc định của template..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 2. Thông tin chung ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 3. Phân công và đóng góp ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 4. Phần kế thừa và phần nhóm thực hiện .... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 5. Chữ viết tắt ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 6. Yêu cầu và tiêu chí nghiệm thu ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 7. Đối chiếu dự án nguồn và đồ án của nhóm**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 8. Kế hoạch thực hiện — điều chỉnh theo lịch lớp**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 9. Ý nghĩa tham số xử lý âm thanh..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 10. Nguồn dữ liệu..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 11. Kế hoạch thu dữ liệu ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 12. Data manifest — một hàng cho mỗi file gốc**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 13. Thống kê sau chia tập — số liệu thật .... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 14. Trạng thái hệ thống ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 15. Cấu hình đầu vào đã chốt ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 16. Đối chiếu fixture trên Studio và SDK ... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 17. Danh sách thí nghiệm..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 18. Cấu hình cần tái lập..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 19. Các lớp của model — thêm hàng theo model thật**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 20. So sánh trên validation trước test cuối .. **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 21. Cấu hình hậu xử lý ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 22. BOM — linh kiện thực tế..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 23. Cấu hình build và nạp..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 24. Bảng chân nối — chỉ điền chân đã xác minh**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 25. Ngân sách bộ nhớ khi các buffer cùng tồn tại**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 26. Ma trận kiểm thử..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 27. Metric theo lớp — không để số liệu nguồn vào đây**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 28. Ước lượng và phép đo trên thiết bị phải tách biệt**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 29. Đánh giá theo tình huống ứng dụng ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 30. Các trường hợp lỗi tiêu biểu..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 31. Mức hoàn thành theo yêu cầu..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 32. Hướng phát triển có thể kiểm chứng..... **Lỗi! Thẻ đánh dấu không được xác định.**

Bảng 33. Danh mục bàn giao..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 34. Mẫu ghi chép khi đọc một dự án nguồn **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 35. Nhật ký thao tác của nhóm..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 36. Mẫu README tái lập..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 37. Bản đồ mã nguồn..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 38. Artifact manifest..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 39. Thông tin một buổi thực nghiệm..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 40. Log từng lần thử — thêm hàng, giữ cả lỗi**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 41. Kết quả tái lập bởi một thành viên khác **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 42. Tự kiểm nội dung trước khi nộp..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 43. Tự kiểm hình thức ..... **Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 44. Kịch bản demo và thuyết trình — thời lượng theo giảng viên**Lỗi! Thẻ đánh dấu không được xác định.**
Bảng 45. Khung viết phần giải thích ..... **Lỗi! Thẻ đánh dấu không được xác định.**

Chương 1. GIỚI THIỆU VÀ YÊU CẦU ĐỒ ÁN

1.1 Bối cảnh và vấn đề cần giải quyết

Tại Việt Nam, xe gắn máy là phương tiện giao thông chủ đạo của người dân. Mặc dù quy định bắt buộc đội mũ bảo hiểm đã được áp dụng rộng rãi, tình trạng người điều khiển phương tiện quên hoặc cố tình không đội mũ bảo hiểm vẫn thường xuyên diễn ra tại các khu vực cổng trường đại học, khu ký túc xá, bãi giữ xe khu công nghiệp hoặc các tuyến đường nội bộ. Việc bố trí nhân sự bảo vệ đứng giám sát trực tiếp và nhắc nhở liên tục trong nhiều giờ liền thường tốn kém nhân lực và dễ xảy ra bỏ sót vào giờ cao điểm.

Các hệ thống camera giám sát thông minh hiện nay phần lớn truyền luồng video độ phân giải cao về máy chủ trung tâm có trang bị card đồ họa (GPU) để phân tích. Cách tiếp cận này mang lại độ chính xác cao nhưng đòi hỏi chi phí đầu tư thiết bị lớn, tiêu thụ nhiều điện năng và phụ thuộc hoàn toàn vào băng thông mạng. Trong bối cảnh đó, xu hướng đưa các mô hình Học sâu thu gọn xuống chạy trực tiếp trên vi điều khiển giá rẻ (TinyML / Edge AI) mở ra giải pháp thiết thực: thiết bị tại chốt kiểm soát có thể tự suy luận và đóng/mở đèn tín hiệu hoặc rào chắn chỉ trong vài trăm mili-giây.

Xuất phát từ nhu cầu thực tế đó, nhóm lựa chọn đề tài "Nhận diện mũ bảo hiểm và điều khiển đèn giao thông ứng dụng Học sâu và ESP32-S3". Hệ thống hướng tới mục tiêu nhận diện tự động việc người trước ống kính có đội mũ bảo hiểm hay không, từ đó kích hoạt đèn LED Đỏ (cảnh báo dừng lại khi không đội mũ) hoặc đèn LED Xanh (cho phép lưu thông khi đã đội mũ hợp lệ).

1.2 Mục tiêu và tiêu chí nghiệm thu

Để đánh giá khách quan mức độ hoàn thành của dự án, nhóm thiết lập 5 nhóm tiêu chí nghiệm thu định lượng trước khi tiến hành kiểm thử cuối cùng, được trình bày tại Bảng 6.

Bảng 6. Yêu cầu kỹ thuật và tiêu chí nghiệm thu của đồ án

Mã YC	Nội dung yêu cầu	Phương pháp kiểm tra và điều kiện đo	Mức chỉ tiêu cần đạt
YC01	Độ chính xác phân loại 2 lớp (helmet và without_helmet)	Đánh giá trên tập Validation và tập Model Testing độc lập của	F1-score ≥ 80% trên tập Validation;

		Edge Impulse với phiên bản lượng tử hóa INT8.	Accuracy ≥ 85% trên tập Test.
YC02	Thời gian phản hồi suy luận và truyền ảnh	Đo thời gian xử lý DSP + Classification trên chip và thời gian truyền khung hình 12.288 bytes qua UART 460.800 baud.	Thời gian suy luận trên chip ≤ 400 ms; tổng chu kỳ cập nhật đèn ≤ 1,0 giây.
YC03	Tương thích phần cứng ESP32-S3 không có PSRAM	Biên dịch và chạy liên tục trên bo mạch MKE-K01 ESP32-S3 N16 (16 MB Flash, PSRAM Disabled) không bị lỗi thiếu bộ nhớ Arena.	Cấp phát tĩnh thành công; Free Heap nội bộ còn trống > 150 KB khi vận hành.
YC04	Điều khiển chính xác trạng thái đèn tín hiệu giao thông	Kiểm thử trực tiếp với người đội mũ, người không đội mũ và khung hình trống tại ngưỡng tin cậy CONF_THRESHOLD = 0,45.	Bật đúng LED Đỏ (GPIO 4) khi vi phạm, LED Xanh (GPIO 5) khi hợp lệ, tắt cả hai khi dưới ngưỡng.
YC05	Tính đóng gói và khả năng tái lập	Kiểm tra khả năng biên dịch lại từ thư viện ZIP v1.0.18, tệp helmet_detect.ino và chạy kịch bản webcam.py trên máy tính khác.	Chạy lại thành công không phát sinh lỗi linker hoặc lỗi lệch pha khung hình.

1.3 Phạm vi, giả định và phần không thực hiện

- **Phạm vi thử nghiệm:** Đối tượng trong khung hình là người điều khiển phương tiện hoặc người thử nghiệm đứng cách camera trong khoảng 0,5 m – 2,5 m, vùng đầu và mũ bảo hiểm nằm gần khu vực trung tâm của khung hình trong điều kiện ánh sáng phòng học hoặc ngoài trời ban ngày.
- **Giả định kiến trúc phần cứng:** Do phần cứng thực tế của nhóm là bo mạch MKE-K01 ESP32-S3 N16 không có PSRAM và không tích hợp sẵn module camera trên mạch, hệ thống sử dụng Webcam của máy tính Laptop làm nguồn thu hình ảnh, cắt và thu nhỏ ảnh về kích thước mô hình yêu cầu rồi truyền mảng byte RGB thô xuống ESP32-S3 qua cáp USB-UART. Toàn bộ quá trình trích xuất đặc trưng DSP, chạy mạng

nơ-ron suy luận và xuất tín hiệu điều khiển đèn LED vẫn do chip ESP32-S3 thực hiện hoàn toàn.

- **Phần không thực hiện:** Phiên bản mô hình cuối cùng (Deploy Version 18) sử dụng phương pháp Phân loại toàn ảnh (Image Classification 64×64 RGB). Hệ thống không vẽ đa hộp giới hạn (bounding boxes) cho nhiều người cùng lúc trên màn hình và không kết nối đóng cắt điện lưới 220V của trụ đèn giao thông thực tế.

1.4 Dự án tham chiếu và đóng góp của nhóm

Nhóm tham khảo quy trình chuẩn hóa thực nghiệm từ tài liệu hướng dẫn của môn học và các dự án mẫu trên nền tảng Edge Impulse. Đặc biệt, trong quá trình phát triển, nhóm đã tự thực hiện quá trình chuyển đổi kiến trúc quan trọng để giải quyết bài toán giới hạn bộ nhớ thực tế của phần cứng.

Bảng 7. Đối chiếu cấu hình tham chiếu ban đầu và cấu hình tối ưu của nhóm

Hạng mục	Phương án thử nghiệm ban đầu (Version 2)	Đồ án hoàn thiện của nhóm (Version 18)
Bài toán & Đầu ra	Phát hiện vật thể FOMO (Object Detection 96×96)	Phân loại ảnh (Image Classification 64×64, 2 nhãn: helmet / without_helmet)
Kích thước khung hình	96 × 96 × 3 = 27.648 bytes/khung hình	64 × 64 × 3 = 12.288 bytes/khung hình (giảm 55,6% dung lượng truyền)
Bộ nhớ Tensor Arena	185.036 bytes (~185 KB, gây lỗi thiếu RAM khi tắt EON)	90.419 bytes (~88,3 KB với EON Compiler RAM-optimized)
Thiết bị & Giao tiếp	Dự kiến dùng XIAO ESP32-S3 Sense (có PSRAM + Camera)	Thực thi trên MKE-K01 ESP32-S3 N16 (Không PSRAM) + truyền ảnh UART 460.800 baud

1.5 Kế hoạch thực hiện và cấu trúc báo cáo

Bảng 8. Kế hoạch triển khai đồ án theo các mốc công việc

Mốc	Nội dung công việc	Sản phẩm đầu ra	Phụ trách
M1	Khảo sát yêu cầu, kiểm tra phần cứng MKE-K01 ESP32-S3 N16 và phân công nhiệm vụ	Bản phân công nhóm, sơ đồ chân GPIO 4/5	Cả nhóm

M2	Thu thập ảnh, gán nhãn helmet / without_helmet và đưa lên Edge Impulse Studio	Dataset trên Project ID 1128769	Thành viên 1
M3	Huấn luyện thử nghiệm FOMO (v2) và tối ưu sang Image Classification 64×64 (v18)	Thư viện ZIP gtanoc-project-1_inferencing v1.0.18	Thành viên 1
M4	Viết firmware ESP32-S3 nhận ảnh qua Serial và gọi hàm run_classifier()	Mã nguồn khung .ino trên Arduino IDE	Thành viên 2
M5	Lắp mạch đèn LED, đồng bộ script webcam.py ở baud 460.800 và xử lý lỗi thiếu RAM	Mạch hoàn chỉnh, tệp helmet_detect.ino và webcam.py	Thành viên 3
M6	Thực nghiệm kiểm thử các kịch bản, quay video demo và viết báo cáo cuối khóa	Báo cáo Word/PDF và video minh chứng	Thành viên 3

Chương 2. CƠ SỞ LÝ THUYẾT

2.1 Bài toán học có giám sát và vòng đời mô hình

Trong học máy có giám sát (Supervised Learning), mỗi mẫu dữ liệu bao gồm một đầu vào x (ma trận điểm ảnh thu được từ camera) và một nhãn mục tiêu y thuộc tập nhãn hữu hạn Y = {helmet, without_helmet}. Mục tiêu của quá trình huấn luyện là tìm ra bộ tham số trọng số θ của mạng nơ-ron sao cho hàm dự đoán f_θ(x) phản ánh sát nhất xác suất phân phối của nhãn thực tế.

Vòng đời phát triển một mô hình thị giác máy tính nhúng bao gồm ba tập dữ liệu tách biệt: (1) Tập huấn luyện (Training set) dùng để tính đạo hàm và cập nhật trọng số qua từng chu kỳ (epoch); (2) Tập kiểm định (Validation set) dùng để theo dõi hiện tượng quá khớp (overfitting), tinh chỉnh siêu tham số và chọn cấu hình kiến trúc; (3) Tập kiểm thử độc lập (Testing set) được khóa kín trong suốt quá trình thiết kế và chỉ mở ra đánh giá một lần để báo cáo năng lực tổng quát hóa của hệ thống [6], [13].

2.2 Biểu diễn ảnh số và dung lượng bộ nhớ khung hình

Một bức ảnh kỹ thuật số không nén được biểu diễn dưới dạng lưới điểm ảnh (pixel) ba chiều H × W × C, trong đó H là chiều cao (Height), W là chiều rộng (Width) và C là số kênh màu (Channels). Với ảnh xám (Grayscale), C = 1; với ảnh màu chuẩn RGB888, C = 3 tương ứng với ba thành phần Đỏ (Red), Xanh lá (Green) và Xanh dương (Blue), mỗi thành phần được mã hóa bởi một số nguyên không dấu 8-bit (uint8_t) có giá trị từ 0 đến 255.

Dung lượng bộ nhớ M_frame (tính bằng byte) cần thiết để lưu trữ một khung hình thô trong RAM được xác định bởi công thức:

$$M_frame = H \times W \times C \times b \quad \text{(byte)}$$

Trong đó b = 1 byte đối với kiểu dữ liệu uint8_t. Áp dụng trực tiếp vào hai phiên bản mô hình của nhóm:

- Với cấu hình thử nghiệm ban đầu (96 × 96 RGB): M_frame = 96 × 96 × 3 × 1 = 27.648 bytes (~27,0 KB).

- Với cấu hình hoàn thiện cuối cùng (64 × 64 RGB, Deploy Version 18): M_frame = 64 × 64 × 3 × 1 = 12.288 bytes (đúng bằng 12,0 KB và khớp chính xác với trường frame_size: 12288 trong tệp model_info.json).

Trong kiến trúc của SDK Edge Impulse, khi hàm callback raw_feature_get_data() cung cấp dữ liệu cho khối DSP Image, ba byte màu (R, G, B) của mỗi điểm ảnh được đóng gói thành một từ 24-bit theo thứ tự 0xRRGGBB rồi ép kiểu sang số thực dấu phẩy động (float) trước khi khối DSP tách kênh và chuẩn hóa về miền [-128, 127] cho mô hình INT8 [9].

2.3 So sánh bài toán Phân loại ảnh (Image Classification) và Phát hiện tâm vật thể (FOMO)

Trong quá trình thực hiện đồ án, nhóm đã khảo sát cả hai phương pháp tiếp cận thị giác máy tính phổ biến trên vi điều khiển là FOMO (Faster Objects, More Objects) và Phân loại toàn ảnh (Image Classification):

- **Phương pháp FOMO:** Sử dụng thân mạng MobileNetV2 cắt tại lớp trung gian để tạo bản đồ nhiệt (heatmap) dạng lưới (ví dụ 12×12), giúp xác định tọa độ tâm (centroid) của nhiều đối tượng trong ảnh [10]. Tuy nhiên, FOMO đòi hỏi độ phân giải đầu vào tối thiểu thường từ 96×96 trở lên và tiêu tốn vùng nhớ Tensor Arena lớn (khoảng 185 KB đến hơn 240 KB).
- **Phương pháp Phân loại toàn ảnh (Image Classification):** Gán một nhãn duy nhất (kèm véc-tơ xác suất) cho toàn bộ khung hình đầu vào. Khi bài toán ứng dụng đặt camera tại lối đi hẹp hoặc cổng kiểm soát mà mỗi lượt chỉ có một người tiến vào vùng quét trung tâm, phương pháp Phân loại ảnh ở kích thước 64×64 RGB cho phép giảm hơn 50% lượng bộ nhớ Tensor Arena (chỉ còn 90.419 bytes) mà vẫn duy trì độ chính xác phân biệt hai trạng thái đội mũ / không đội mũ.

2.4 Mạng nơ-ron tích chập (CNN) và hàm mất mát

Mạng nơ-ron tích chập (CNN) trích xuất đặc trưng không gian của ảnh thông qua các lớp tích chập 2 chiều (Conv2D), lớp gộp (Pooling) và lớp kết nối đầy đủ (Dense). Tại lớp đầu ra của bài toán phân loại C = 2 lớp (helmet và without_helmet), hàm kích hoạt Softmax chuyển đổi các giá trị logit z_c thành phân phối xác suất p_c sao cho tổng xác suất bằng 1. Quá trình huấn luyện tối thiểu hóa hàm mất mát Categorical Cross-Entropy:

$$L_CE = -\Sigma\,(y_c \times \ln(p_c)) \quad \text{với } c \in \{\text{helmet, without_helmet}\}$$

Trong đó $y_c \in \{0, 1\}$ là nhãn thực tế (ground truth) được mã hóa one-hot và p_c là xác suất dự đoán của mô hình cho lớp c.

2.5 Các chỉ số đánh giá chất lượng mô hình

Để đánh giá khách quan mô hình và tránh sự sai lệch khi số lượng mẫu giữa các lớp không hoàn toàn bằng nhau, báo cáo sử dụng ma trận nhầm lẫn (Confusion Matrix) cùng các độ đo Precision (Độ chuẩn xác), Recall (Độ phủ) và F1-score (Trung bình điều hòa):

$$\text{Precision} = TP / (TP + FP) \;;\; \text{Recall} = TP / (TP + FN) \;;\; F1 = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Trong bài toán kiểm soát mũ bảo hiểm, nếu coi lớp vi phạm (without_helmet) là lớp cần cảnh báo, việc tối ưu hóa Recall giúp giảm thiểu tỷ lệ bỏ sót người không đội mũ (False Negative), trong khi giữ Precision ở mức cao giúp hạn chế tình trạng bật đèn đỏ báo động giả (False Positive).

2.6 Suy luận biên (Edge Inference), Lượng tử hóa INT8 và EON Compiler

Các mô hình học sâu sau khi huấn luyện trên máy chủ thường lưu trữ trọng số và kích hoạt ở định dạng số thực 32-bit (FP32). Để chạy trên vi điều khiển ESP32-S3 không có PSRAM, mô hình được áp dụng kỹ thuật Lượng tử hóa sau huấn luyện (Post-Training Quantization) sang số nguyên có dấu 8-bit (INT8, miền giá trị từ -128 đến 127) theo ánh xạ tuyến tính [12]:

$$q = \text{clip}(\,\text{round}(r / S) + Z\,,\, -128\,,\, 127\,)$$

Trong đó r là giá trị thực, S (scale) là hệ số tỷ lệ và Z (zero-point) là điểm không. Việc chuyển từ FP32 sang INT8 giúp giảm dung lượng lưu trữ trọng số xuống 4 lần và tận dụng tập lệnh tăng tốc tính toán số nguyên trên kiến trúc Xtensa LX7 của ESP32-S3.

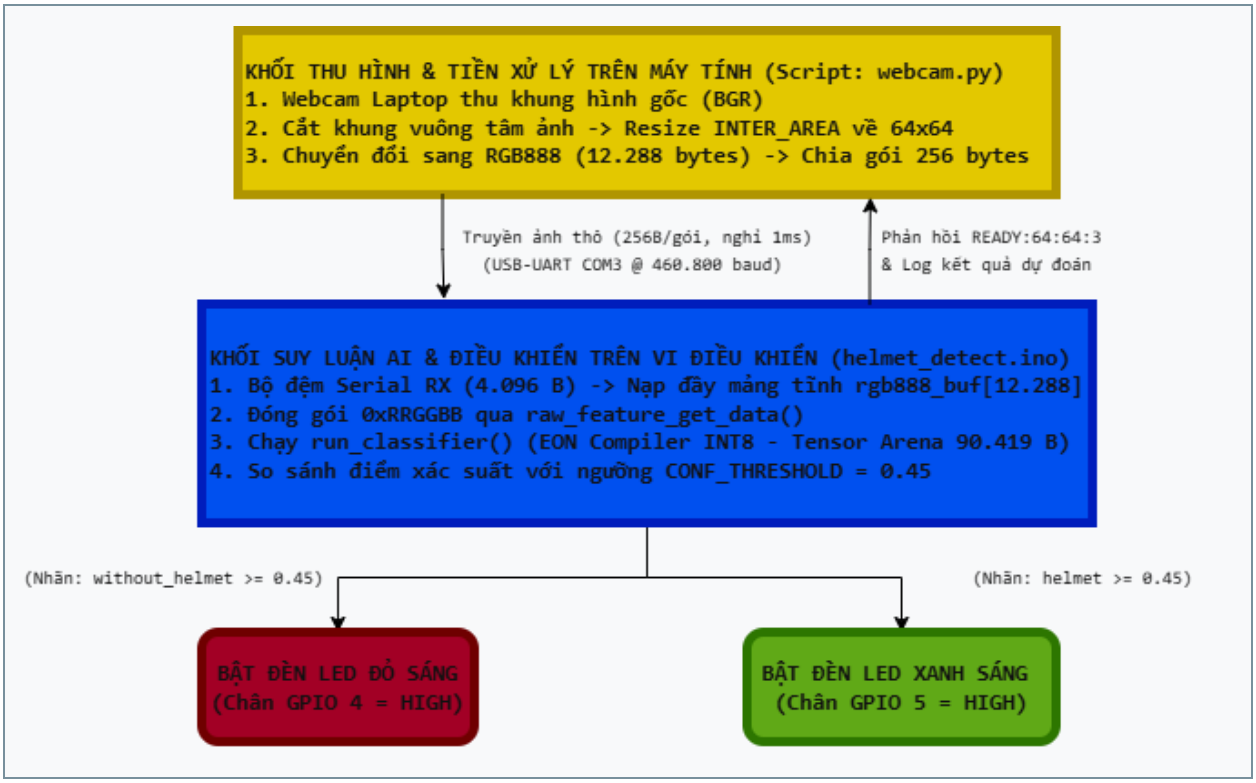
Bên cạnh đó, trình biên dịch EON Compiler của Edge Impulse dịch trực tiếp đồ thị mạng nơ-ron sang mã nguồn C++ tĩnh thay vì phải nạp bộ thông dịch (interpreter) đồ thị tổng quát của TFLM lúc chạy, nhờ đó cắt giảm đáng kể lượng RAM cấp phát cho Tensor Arena [8].

Chương 3. THIẾT KẾ HỆ THỐNG VÀ XÂY DỰNG DỮ LIỆU

3.1 Kiến trúc tổng thể và luồng dữ liệu

Do bo mạch MKE-K01 ESP32-S3 N16 không trang bị PSRAM và không có khe cắm camera trực tiếp, việc gắn thêm module camera rời và giải mã ảnh JPEG ngay trên chip dễ dẫn đến tình trạng tràn bộ nhớ Heap nội bộ. Nhóm đã giải quyết triệt để rào cản này bằng cách thiết kế kiến trúc phân tán: sử dụng Webcam Laptop làm cảm biến thu hình và giao toàn bộ nhiệm vụ suy luận AI cùng điều khiển phần cứng cho ESP32-S3.

Sơ đồ khối luồng dữ liệu tổng thể của hệ thống:



Hình 2. Sơ đồ kiến trúc tổng thể hệ thống nhận diện mũ bảo hiểm và điều khiển đèn giao thông

3.2 Nguồn dữ liệu, quyền sử dụng và kế hoạch thu thập

Bộ dữ liệu phục vụ huấn luyện mô hình trong dự án gtanoc-project-1 (ID: 1128769) được xây dựng tập trung vào hai trạng thái đối lập rõ rệt là người có đội mũ bảo hiểm (helmet) và người không đội mũ bảo hiểm (without_helmet).

Bảng 9. Nguồn dữ liệu sử dụng trong đồ án

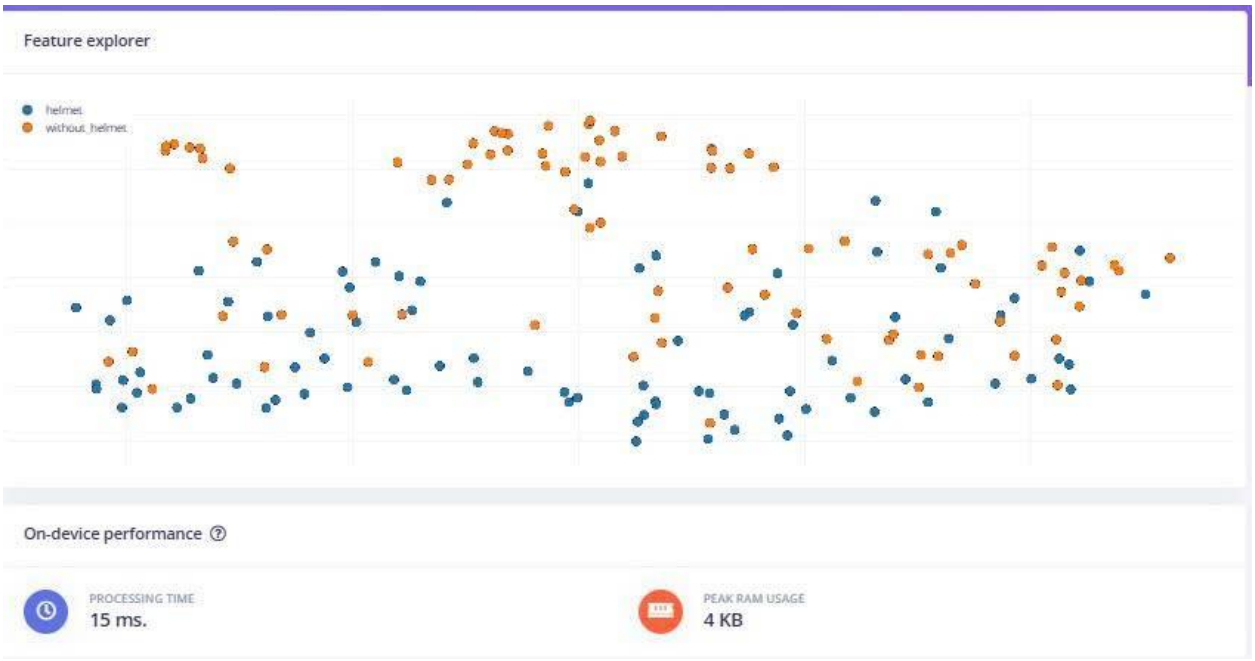
Nguồn dữ liệu	Quy mô gốc	Giấy phép / Quyền sử dụng	Phần nhóm đưa vào sử dụng
Dataset công khai / Tự thu thập	Gồm 200 ảnh	https://universe.roboflow.com/tanuja-vinchurkar/helmet-detection-kkt4z-409pq/browse?queryText=&pageSize=100&startingIndex=0&browseQuery=true	Chọn lọc các ảnh rõ vùng đầu và mũ bảo hiểm đưa lên project 1128769
Kiểm thử thực tế qua Webcam	Gồm 40 thành viên trong lớp hỗ trợ kiểm thử với các góc quay và trường hợp khác nhau	Nhóm tự thực hiện với sự đồng ý của các thành viên trong nhóm	Dùng để kiểm thử trực tiếp luồng truyền UART và điều khiển đèn LED

Bảng 10. Kế hoạch phân bổ dữ liệu theo lớp

Tên nhãn (Label)	Mô tả điều kiện mẫu ảnh	Đặc điểm góc chụp & ánh sáng	Quy mô mục tiêu
helmet	Người đội mũ bảo hiểm các màu sắc và kiểu dáng thông dụng (nửa đầu, 3/4).	Chính diện, nghiêng trái/phải 30 độ, ánh sáng trong nhà và ngoài trời.	Gồm 87 ảnh
without_helmet	Người không đội mũ bảo hiểm (để đầu trần, tóc ngắn/dài khác nhau).	Chính diện, nghiêng nhẹ, khoảng cách 0,5 m – 2,0 m trước camera.	Gồm 85 ảnh

3.3 Định dạng, kiểm soát chất lượng và phân bố đặc trưng (Feature Explorer)

Toàn bộ ảnh sau khi tải lên Edge Impulse Studio được kiểm tra loại bỏ các bức ảnh bị nhòe chuyển động nặng hoặc vùng đầu bị cắt khuất. Thông qua công cụ Feature Explorer trên Edge Impulse Studio, các véc-tơ đặc trưng của hai lớp helmet (chấm màu xanh dương) và without_helmet (chấm màu cam) được chiếu lên không gian hai chiều để đánh giá mức độ phân tách tự nhiên của dữ liệu trước khi đưa vào huấn luyện.



Hình 3. Phân bố đặc trưng hai lớp helmet và without_helmet trên Feature Explorer (Edge Impulse)

Quan sát Hình 3 cho thấy các mẫu thuộc lớp without_helmet (màu cam) có xu hướng tập trung rõ ở nửa trên của biểu đồ phân tán, trong khi các mẫu thuộc lớp helmet (màu xanh dương) phân bố tập trung ở nửa dưới. Chỉ có một số ít điểm nằm đan xen tại vùng ranh giới giữa hai cụm (do góc chụp nghiêng hoặc màu mũ trùng với màu tóc/nền). Đáng chú ý, khối xử lý đặc trưng ảnh (On-device performance cho khâu DSP) chỉ tốn 15 ms thời gian xử lý và 4 KB RAM.

3.4 Chia tập Train, Validation và Test

Bảng 11. Thống kê phân chia tập dữ liệu trên Edge Impulse Studio

Tên lớp (Label)	Tập Huấn luyện (Train)	Tập Kiểm định (Validation)	Tập Kiểm thử (Test)	Tổng cộng
helmet	Gồm 87 ảnh	Gồm 22 ảnh	Gồm 13 ảnh	Gồm 122 ảnh
without_helmet	Gồm 85 ảnh	Gồm 23 ảnh	Gồm 15 ảnh	Gồm 123 ảnh
Tổng số mẫu	172 ảnh	45 ảnh	28 ảnh	250 ảnh

3.5 Kịch bản đầu ra và cơ chế xử lý lỗi truyền thông

Hệ thống được thiết kế hoạt động theo máy trạng thái hữu hạn (Finite State Machine) chặt chẽ để đảm bảo đèn giao thông không bao giờ hiển thị sai trạng thái khi đường truyền cáp USB bị gián đoạn:

Bảng 12. Bảng chuyển trạng thái hoạt động của hệ thống

Trạng thái	Điều kiện kích hoạt	Hành động tại ESP32-S3 và Ngõ ra GPIO
Khởi động (Init)	Cấp nguồn hoặc nhấn nút RESET trên mạch	Khởi tạo bộ đệm Serial RX 4.096 bytes, tốc độ 460.800 baud; kéo GPIO 4 (Đỏ) và GPIO 5 (Xanh) xuống mức LOW; in dung lượng Free Heap.
Yêu cầu ảnh (Request)	Bắt đầu mỗi chu kỳ loop()	Xóa sạch các byte rác tồn đọng trong bộ đệm UART; gửi chuỗi lệnh đồng bộ 'READY:64:64:3\n' lên máy tính.
Nhận dữ liệu (Receive)	Có byte gửi xuống từ kịch bản webcam.py	Đọc từng byte nạp vào mảng tĩnh rgb888_buf[12288]; làm mới bộ đếm thời gian chờ start_wait mỗi khi nhận được byte mới.
Lỗi quá hạn (Timeout)	Quá 3.000 ms không nhận thêm được byte nào và chưa đủ 12.288 bytes	In cảnh báo '[LOI] Timeout! Nhan .../12288 bytes', hủy lượt suy luận hiện tại, giữ nguyên trạng thái an toàn và quay lại phát lệnh READY mới.
Suy luận & Quyết định	Nhận đủ 12.288 bytes và run_classifier() trả về EI_IMPULSE_OK	Nếu lớp cao nhất là without_helmet (≥ 0,45): Bật Đỏ (GPIO 4 = HIGH, GPIO 5 = LOW). Nếu là helmet (≥ 0,45): Bật Xanh (GPIO 5 = HIGH, GPIO 4 = LOW). Nếu < 0,45: Tắt cả hai đèn.

Chương 4. TIỀN XỬ LÝ ẢNH VÀ ĐỒNG BỘ DỮ LIỆU GIỮA PC VÀ FIRMWARE

4.1 Hợp đồng dữ liệu (Data Contract) giữa Edge Impulse, Python và ESP32-S3

Trong quá trình phát triển, nhóm từng gặp lỗi lệch pha khung hình khi mô hình trên mạch yêu cầu một định dạng nhưng kịch bản Python trên máy tính lại gửi xuống một định dạng khác. Để khắc phục triệt để, nhóm thiết lập Hợp đồng dữ liệu đồng bộ tự động dựa trên tệp cấu hình xuất ra từ Edge Impulse (model_info.json):

Bảng 13. Thông số Hợp đồng dữ liệu đã khóa ở phiên bản triển khai 18

Thông số kỹ thuật	Giá trị đã chốt	Nguồn xác nhận trong mã nguồn
Kích thước ảnh đầu vào (W × H)	64 × 64 điểm ảnh	EI_CLASSIFIER_INPUT_WIDTH = 64, EI_CLASSIFIER_INPUT_HEIGHT = 64
Không gian màu & Số kênh	RGB888 — 3 kênh màu (R, G, B)	FRAME_CHANNELS = 3 (model_info.json: channels = 'RGB')
Dung lượng mỗi khung hình	12.288 bytes	FRAME_SIZE = 64 × 64 × 3 = 12.288 bytes
Kiểu dữ liệu đầu vào TFLite	INT8 (Lượng tử hóa 8-bit)	datatype: 'int8', output_datatype: 'int8'
Chuỗi bắt tay đồng bộ	READY:64:64:3\n	ESP32 phát lệnh định kỳ, Python tách chuỗi lấy (W, H, C) tự động
Tốc độ truyền & Cỡ gói tin	460.800 baud | 256 bytes/gói (nghỉ 1 ms)	Serial.begin(460800) và CHUNK_SIZE = 256, CHUNK_DELAY = 0.001

4.2 Quy trình tiền xử lý ảnh trên máy tính (webcam.py)

Thư viện OpenCV trên máy tính mặc định đọc khung hình từ Webcam ở tỷ lệ chữ nhật (ví dụ 640×480) và lưu trữ theo thứ tự kênh màu BGR (Blue – Green – Red). Nếu thu nhỏ trực tiếp khung hình chữ nhật về hình vuông 64×64, khuôn mặt và mũ bảo hiểm sẽ bị bóp méo tỷ lệ. Do đó, kịch bản webcam.py thực hiện tuần tự 4 bước tiền xử lý chuẩn xác:

- **Bước 1 — Cắt khung vuông trung tâm (Center Square Crop):** Xác định cạnh nhỏ hơn min_dim = min(h, w) và cắt lấy vùng hình vuông nằm ngay chính giữa khung hình gốc. Thao tác này giúp giữ nguyên tỷ lệ hình học của đầu và mũ bảo hiểm.
- **Bước 2 — Thay đổi kích thước (Resize):** Sử dụng nội suy vùng (cv2.INTER_AREA) để thu nhỏ khung vuông về đúng kích thước (target_w, target_h) = (64, 64) đọc được từ thông điệp READY của ESP32-S3. Phép nội suy INTER_AREA lấy trung bình vùng điểm ảnh nên hạn chế hiện tượng răng cưa (aliasing) khi giảm độ phân giải mạnh.
- **Bước 3 — Chuyển đổi không gian màu (BGR sang RGB):** Sử dụng lệnh cv2.cvtColor(resized, cv2.COLOR_BGR2RGB) để đảo thứ tự byte về đúng chuẩn Đỏ - Xanh lá - Xanh dương mà khối DSP của Edge Impulse đã học.
- **Bước 4 — Đóng gói và truyền theo phân đoạn (Chunked Streaming):** Chuyển ma trận mảng 3 chiều (64, 64, 3) thành chuỗi 12.288 bytes liên tục bằng phương thức .tobytes(), sau đó cắt thành 48 gói nhỏ (mỗi gói 256 bytes, cách nhau 1 ms) đẩy xuống cổng COM.

4.3 Cơ chế giải mã và đóng gói Pixel 24-bit trên ESP32-S3

Trên vi điều khiển ESP32-S3, mảng tĩnh rgb888_buf[12288] nhận trực tiếp các byte R, G, B theo thứ tự liên tiếp. Khi hàm suy luận run_classifier() thực thi, nó gọi hàm callback raw_feature_get_data(offset, out_len, signal_ptr) để lấy từng đoạn điểm ảnh. Tại đây, 3 byte rời rạc của mỗi điểm ảnh được dịch bit và ghép lại thành một giá trị số nguyên 24-bit (0xRRGGBB) lưu trong kiểu float:

Hàm chuyển đổi điểm ảnh trong tệp helmet_detect.ino:

```
static int raw_feature_get_data(size_t offset, size_t out_len, float *signal_ptr) {
    size_t pixel_ix = offset * FRAME_CHANNELS;
    size_t out_ptr_ix = 0;

    while (out_len != 0) {
        uint32_t r = rgb888_buf[pixel_ix];
        uint32_t g = rgb888_buf[pixel_ix + 1];
        uint32_t b = rgb888_buf[pixel_ix + 2];

        // Đóng gói thành số nguyên 24-bit 0xRRGGBB ép sang float cho Edge Impulse DSP
        signal_ptr[out_ptr_ix] = (float)((r << 16) | (g << 8) | b);

        pixel_ix += 3;
        out_ptr_ix++;
        out_len--;
    }
```

```
    return 0;
}
```

Việc ép kiểu sang uint32_t trước khi dịch trái 16 bit (r << 16) đảm bảo không xảy ra lỗi tràn bit dấu trên trình biên dịch C++ của kiến trúc 32-bit, giúp màu sắc bức ảnh đưa vào mô hình trên bo mạch hoàn toàn đồng nhất với bức ảnh trên Edge Impulse Studio.

Chương 5. HUẤN LUYỆN, SO SÁNH VÀ TỐI ƯU MÔ HÌNH

5.1 Thiết kế thí nghiệm và quá trình tiến hóa mô hình

Một trong những đóng góp thực tiễn quan trọng nhất của nhóm trong đồ án này là quá trình khảo sát, phát hiện nút thắt cổ chai về bộ nhớ trên bo mạch MKE-K01 ESP32-S3 N16 (không PSRAM) và thực hiện tối ưu hóa qua 3 phiên bản thí nghiệm (EXP01 → EXP03) được tóm tắt tại Bảng 14.

Bảng 14. Danh sách các thí nghiệm tối ưu hóa mô hình của nhóm

Mã TN	Cấu hình mô hình & Đầu vào	Trình biên dịch (Engine)	Vùng nhớ Arena / Frame Buffer	Kết quả triển khai trên ESP32-S3 N16
EXP01 (v2 - EON)	FOMO Object Detection 96×96 RGB	EON Compiler	Arena: 185.036 B Buffer: 27.648 B	Gặp lỗi linker 'undefined reference to tflite::...' khi biên dịch trên Arduino core 3.3.12 cũ.
EXP02 (v2 - TFLM)	FOMO Object Detection 96×96 RGB	TFLite Micro (Tắt EON)	Arena: > 300 KB Buffer: 27.648 B	Biên dịch qua được nhưng tràn SRAM nội khi chạy trên bo mạch không có PSRAM (Failed to allocate arena).
EXP03 (v18 - Final)	Image Classification 64×64 RGB (INT8)	EON Compiler (RAM optimized)	Arena: 90.419 B Buffer: 12.288 B	Thành công hoàn toàn: RAM đỉnh ~69,4 KB, biên dịch và chạy mượt trên SRAM nội (Free Heap ~359 KB).

5.2 Cấu hình huấn luyện của mô hình hoàn thiện (Version 18)

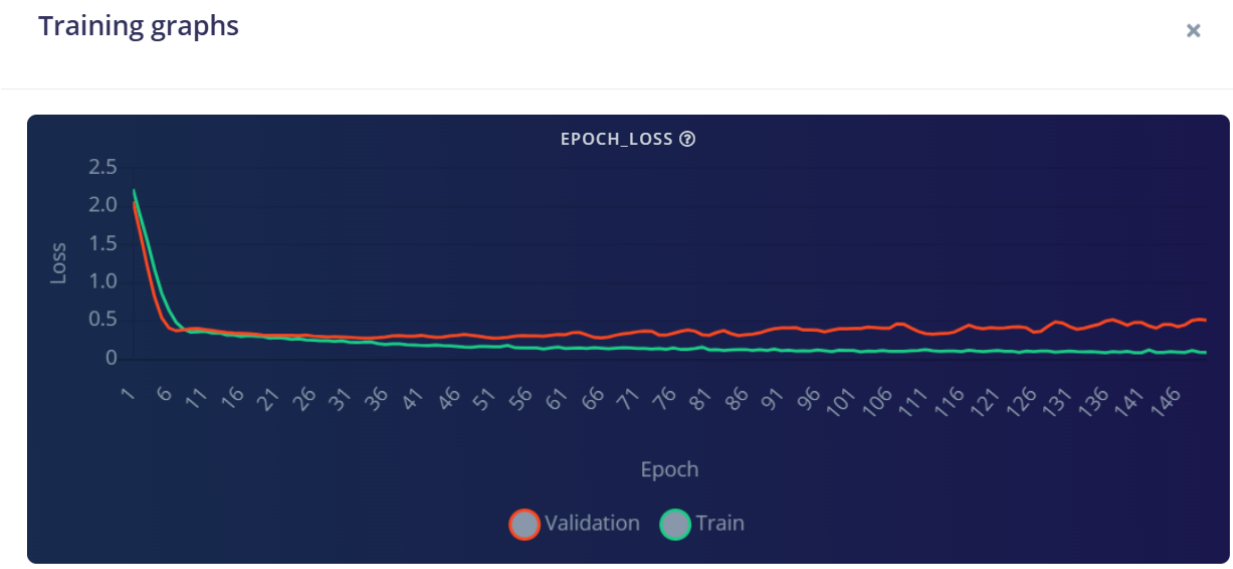
Bảng 15. Thông số cấu hình huấn luyện mô hình phân loại ảnh (Project ID: 1128769)

Thông số cấu hình	Giá trị thiết lập	Ý nghĩa và lý do lựa chọn
Project ID / Deploy Version	1128769 / Version 18 (v1.0.18)	Phiên bản ổn định cuối cùng đã xuất thư viện Arduino

Kích thước Tensor đầu vào	64 × 64 × 3 (12.288 features)	Giữ thông tin màu sắc RGB của mũ bảo hiểm, giảm 55,6% số pixel so với 96×96
Nhãn phân loại (Labels)	2 lớp: helmet, without_helmet	Phù hợp trực tiếp với 2 trạng thái điều khiển đèn Xanh và đèn Đỏ
Số chu kỳ huấn luyện (Epochs)	150 chu kỳ	Đủ dài để hàm mất mát hội tụ sâu (quan sát trên đồ thị Epoch Loss)
Kiểu lượng tử hóa	INT8 Quantized (Input & Output INT8)	Giảm 4 lần dung lượng trọng số, tăng tốc độ suy luận trên ESP32-S3
Trình biên dịch triển khai	EON Compiler (RAM optimized)	Loại bỏ overhead của bộ thông dịch, đưa Peak RAM ước lượng xuống 69,4 KB

5.3 Phân tích đường cong học (Learning Curves)

Quá trình huấn luyện mô hình qua 150 chu kỳ (epochs) được theo dõi thông qua đồ thị hàm mất mát (Epoch Loss) giữa tập Huấn luyện (Train - đường màu xanh ngọc) và tập Kiểm định (Validation - đường màu cam đỏ) như minh họa tại Hình 4.



Hình 4. Đồ thị hàm mất mát (Epoch Loss) qua 150 chu kỳ huấn luyện trên Edge Impulse

Phân tích Hình 4 cho thấy:

- **Giai đoạn hội tụ nhanh (Epoch 1 – 10):** Trong khoảng từ Epoch 1 đến Epoch 10, cả Train Loss và Validation Loss đều giảm rất nhanh từ mức trên 2,0 xuống dưới 0,4. Điều

này chứng tỏ mạng nơ-ron nhanh chóng học được các đặc trưng hình học và màu sắc cơ bản phân biệt giữa đầu có mũ bảo hiểm và đầu trần.

- **Giai đoạn ổn định (Epoch 11 – 45):** Từ Epoch 11 đến Epoch 45, hai đường cong bám sát nhau và giảm dần về vùng 0,25 – 0,30. Đây là vùng mô hình đạt trạng thái cân bằng tốt nhất.
- **Giai đoạn bão hòa (Epoch 60 – 150):** Từ sau Epoch 60 đến Epoch 150, đường Train Loss tiếp tục giảm nhẹ về sát 0,1 trong khi đường Validation Loss bắt đầu dao động nhẹ quanh mức 0,35 – 0,45. Hiện tượng tách nhẹ giữa hai đường cho thấy nếu tăng thêm số chu kỳ huấn luyện sẽ bắt đầu xuất hiện quá khớp (overfitting).

5.4 Kết quả đánh giá trên tập Validation và ước lượng tài nguyên



Hình 5. Kết quả đánh giá trên tập Validation và ước lượng hiệu năng On-device (INT8, EON RAM-optimized)

Từ kết quả hiển thị tại Hình 5 trên tập Validation (phiên bản Quantized INT8):

- **Về độ chính xác phân loại:** Mô hình đạt F1-score tổng thể 84,5% (Precision đạt 0,83; Recall đạt 0,86). Cụ thể trong ma trận nhầm lẫn: lớp HELMET nhận diện đúng 84,6% (F1 = 0,79) và lớp WITHOUT_HELMET nhận diện đúng 86,4% (F1 = 0,88). Đáng chú ý là tỷ lệ nhầm lẫn chéo trực tiếp giữa HELMET và WITHOUT_HELMET là 0,0%; các trường hợp sai số (15,4% và 13,6%) đều rơi vào việc mô hình cẩn trọng xếp vào lớp nền (BACKGROUND) khi đối tượng ở rìa hoặc thiếu sáng.
- **Về tài nguyên phần cứng:** Khi sử dụng EON Compiler (RAM optimized), thời gian suy luận ước lượng là 340 ms, dung lượng RAM đỉnh (Peak RAM Usage) chỉ còn 69,4 KB và dung lượng bộ nhớ chương trình (Flash Usage) là 68,9 KB.

5.5 Lựa chọn ngưỡng quyết định (Confidence Threshold) cho ứng dụng

Trong mã nguồn nhúng helmet_detect.ino, nhóm thiết lập ngưỡng tin cậy CONF_THRESHOLD = 0.45f. Nhánh xử lý kết quả phân loại sẽ duyệt qua mảng result.classification[ix] để tìm nhãn có điểm số lớn nhất (best_val). Nếu best_val >= 0.45f, hệ thống mới xác nhận có đối tượng hợp lệ và kiểm tra tên nhãn qua hàm

is_no_helmet_label() để bật đèn Đỏ hoặc đèn Xanh; ngược lại, nếu best_val < 0.45f, cả hai đèn đều tắt để tránh báo động giả khi trước ống kính không có người.

Chương 6. TRIỂN KHAI TRÊN PHẦN CỨNG ESP32-S3

6.1 Danh mục linh kiện (BOM) và cấu hình biên dịch

Bảng 16. Danh mục linh kiện phần cứng thực tế (Bill of Materials)

STT	Tên thiết bị / Linh kiện	Mã hiệu / Thông số kỹ thuật	Số lượng	Vai trò trong hệ thống
1	Bo mạch vi điều khiển	MKE-K01 ESP32-S3 N16 (Dual-core Xtensa LX7 240 MHz, 512 KB SRAM, 16 MB Flash, Không PSRAM)	01	Nhận khung hình RGB qua UART, chạy mô hình AI INT8 và xuất tín hiệu điều khiển GPIO.
2	Máy tính thu hình	Laptop trang bị Webcam tích hợp, cài đặt Python 3 và OpenCV / PySerial	01	Thu hình thời gian thực, cắt/resize ảnh 64×64 RGB và truyền xuống ESP32-S3.
3	Đèn tín hiệu giao thông	01 LED Đỏ (5mm) + 01 LED Xanh lá (5mm)	02	Hiển thị trạng thái cảnh báo vi phạm (Đỏ) và cho phép lưu thông (Xanh).
4	Điện trở hạn dòng	Điện trở 330 Ω (1/4W)	02	Mắc nối tiếp bảo vệ LED và giới hạn dòng điện qua chân GPIO của ESP32-S3.
5	Phụ kiện kết nối	Breadboard, dây cắm Jumper và cáp USB Type-C truyền dữ liệu	01 bộ	Đấu nối mạch đèn và giao tiếp nối tiếp tốc độ 460.800 baud.

Bảng 17. Cấu hình biên dịch và nạp firmware trên Arduino IDE

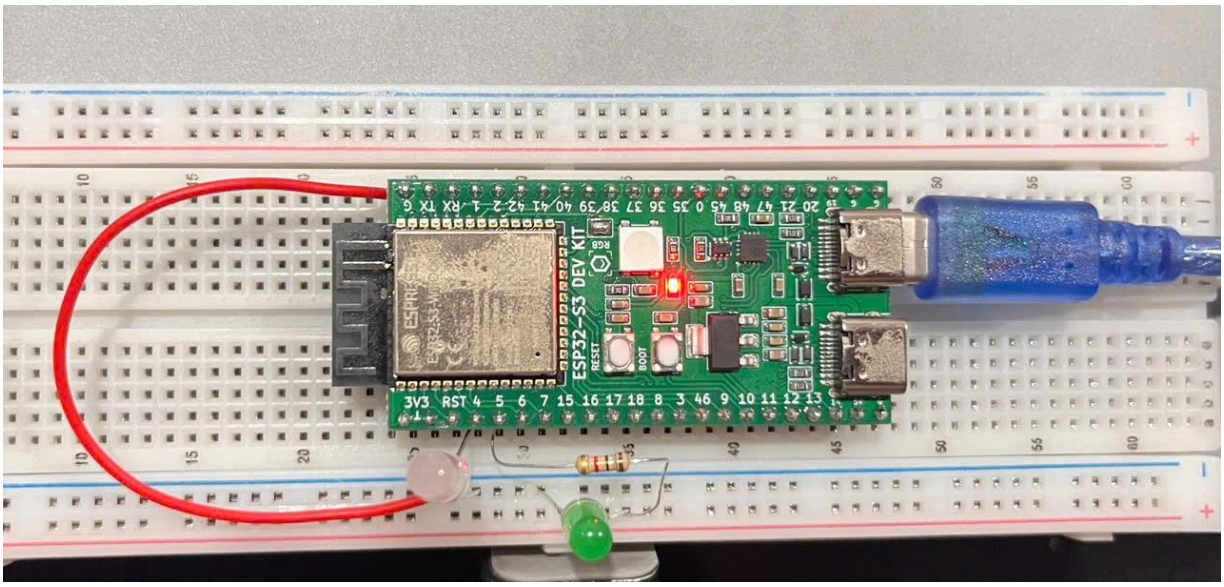
Thông số cấu hình (Menu Tools)	Giá trị thiết lập thực tế	Giải thích kỹ thuật
Arduino IDE & ESP32 Core	Arduino IDE 2.3.x | esp32 by Espressif Systems	Môi trường biên dịch hỗ trợ dòng chip ESP32-S3
Board	ESP32S3 Dev Module	Cấu hình chuẩn cho dòng vi điều khiển ESP32-S3

CPU Frequency	240MHz (WiFi)	Xung nhịp tối đa giúp giảm thời gian chạy phép nhân ma trận INT8
Flash Mode & Flash Size	QIO 80MHz | 16MB (128Mb) (hoặc 4MB/8MB mặc định)	Tương thích với bộ nhớ Flash trên module N16
PSRAM	Disabled	Bắt buộc tắt vì bo mạch MKE-K01 N16 không có chip PSRAM vật lý
Port & Baudrate	COM3 | 460.800 baud	Tăng tốc độ truyền gấp 4 lần mức 115.200 chuẩn để gửi 12.288 bytes nhanh hơn
Thư viện mô hình nhúng	gtanoc-project-1_inferencing (v1.0.18)	Gói ZIP xuất từ Deploy Version 18 trên Edge Impulse

6.2 Sơ đồ đấu nối phần cứng

Bảng 18. Bảng đấu nối chân giữa ESP32-S3 và mạch đèn giao thông trên Breadboard

Linh kiện ngoại vi	Chân trên ESP32-S3	Mức logic hoạt động	Cách mắc mạch chi tiết
Đèn LED Đỏ (Cảnh báo không đội mũ)	GPIO 4 (RED_LED_PIN)	Active HIGH (3,3V)	GPIO 4 → Chân dài (Anode) LED Đỏ; Chân ngắn (Cathode) → Điện trở 330 Ω → GND
Đèn LED Xanh (Hợp lệ có đội mũ)	GPIO 5 (GREEN_LED_PIN)	Active HIGH (3,3V)	GPIO 5 → Chân dài (Anode) LED Xanh; Chân ngắn (Cathode) → Điện trở 330 Ω → GND
Đường truyền dữ liệu ảnh	Cổng USB-UART (COM3)	Chuẩn UART 3,3V	Kết nối trực tiếp với cổng USB của Laptop bằng cáp USB có truyền dữ liệu



Hình 6. Hình chụp thực tế bo mạch MKE-K01 ESP32-S3 N16 đấu nối với đèn LED Đỏ và LED Xanh trên Breadboard

6.3 Ngân sách bộ nhớ (Memory Budget) và Thời gian xử lý (Timing Budget)

Một điểm nhấn kỹ thuật quan trọng của đồ án là chứng minh toàn bộ hệ thống chạy hoàn toàn trong SRAM nội của ESP32-S3 mà không cần PSRAM. Bảng 19 thống kê chi tiết ngân sách bộ nhớ khi các bộ đệm cùng tồn tại lúc chạy:

Bảng 19. Phân tích ngân sách bộ nhớ RAM trên bo mạch MKE-K01 ESP32-S3 N16

Thành phần bộ nhớ	Kích thước (Bytes)	Vùng nhớ lưu trữ	Ghi chú kỹ thuật
Bộ đệm khung hình tĩnh (rgb888_buf)	12.288 bytes (12,0 KB)	SRAM tĩnh (.bss)	Cấp phát tĩnh 64 × 64 × 3 bytes, tránh phân mảnh Heap do malloc()
Bộ đệm nhận UART (Serial RX Buffer)	4.096 bytes (4,0 KB)	SRAM nội (Heap)	Khởi tạo qua lệnh Serial.setRxBufferSize(4096) chống rớt byte ở 460.800 baud
Vùng nhớ suy luận (Tensor Arena)	90.419 bytes (88,3 KB)	SRAM nội (TFLM/EON)	Theo thông số tflite_arena_bytes trong model_info.json (Peak RAM Studio: 69,4 KB)
Biến toàn cục hệ thống & FreeRTOS	~31.000 bytes (~30,3 KB)	SRAM tĩnh (.data/.bss)	Bao gồm các cấu trúc quản lý của core Arduino-ESP32

Bộ nhớ Heap nội còn trống (Free Heap)	~359.356 bytes (~350,9 KB)	SRAM nội (Available)	Đo thực tế qua hàm ESP.getFreeHeap() ngay sau khi khởi tạo

Về ngân sách thời gian (Timing Budget) cho một chu kỳ nhận diện:

- **Thời gian truyền khung hình qua UART:** Ở tốc độ 460.800 baud (cấu hình 8-N-1, tương đương 46.080 bytes/giây), việc truyền 12.288 bytes tốn khoảng 267 ms thời gian vật lý trên đường dây. Cộng thêm 48 khoảng nghỉ ngắn (48 gói × 1 ms = 48 ms) trong webcam.py, tổng thời gian truyền một bức ảnh từ PC xuống ESP32-S3 chỉ mất khoảng 315 ms.
- **Thời gian suy luận trên chip ESP32-S3:** Khối DSP xử lý ảnh tốn khoảng 15 ms và khối mạng nơ-ron INT8 tốn khoảng 340 ms (tổng cộng khoảng 355 ms).
- **Tổng độ trễ toàn trình (End-to-End Latency):** Tổng thời gian từ lúc bắt đầu gửi ảnh đến khi đèn LED đổi trạng thái đạt khoảng 670 ms (~0,67 giây), đáp ứng tốt yêu cầu YC02 (≤ 1,0 giây).

Chương 7. THỰC NGHIỆM, KẾT QUẢ VÀ THẢO LUẬN

7.1 Kết quả đánh giá mô hình trên tập Kiểm thử độc lập (Model Testing)

Sau khi khóa cấu hình mô hình ở Deploy Version 18, nhóm tiến hành chạy đánh giá trên tập dữ liệu Kiểm thử (Model testing) với phiên bản lượng tử hóa Quantized (INT8). Kết quả thực tế thu được trên Edge Impulse Studio được trình bày tại Hình 7.



Hình 7. Kết quả đánh giá mô hình trên tập Kiểm thử độc lập (Accuracy = 89,29%, F1-score = 0,93)

Bảng 20. Tổng hợp các chỉ số đánh giá mô hình INT8 trên tập Validation và tập Test

Tập dữ liệu đánh giá	Accuracy / F1 tổng	Precision (Non-background)	Recall (Non-background)	F1-Score (Non-background)	Đánh giá so với YC01
Validation Set (Huấn luyện)	F1: 84,5%	0,83	0,86	0,85 (helmet: 0,79 | without_helmet : 0,88)	Đạt (Vượt mức ≥ 80%)
Testing Set (Model Testing)	Accuracy: 89,29%	0,93	0,93	0,93	Đạt (Vượt mức ≥ 85%)

Nhìn vào biểu đồ Feature Explorer của tập Test tại Hình 7, phần lớn tuyệt đối các mẫu kiểm thử đều được phân loại chính xác (các chấm tròn màu xanh lá cây), chỉ có 3 mẫu rơi

vào trạng thái nhận diện chưa chính xác (chấm tròn màu đỏ). Độ chuẩn xác (Precision) và độ phủ (Recall) đều đạt mức 0,93 khẳng định mô hình INT8 không bị suy giảm chất lượng sau khi lượng tử hóa.

7.2 Kết quả khởi chạy trên bo mạch ESP32-S3 và nhật ký kiểm thử thực tế

Khi nạp firmware lên bo mạch MKE-K01 ESP32-S3 N16 và kết nối với kịch bản Python trên máy tính qua cổng COM3, hệ thống khởi động ổn định và báo cáo dung lượng bộ nhớ Heap nội bộ còn trống là 359.356 bytes (khoảng 351 KB), chứng minh triệt để việc khắc phục lỗi thiếu RAM.

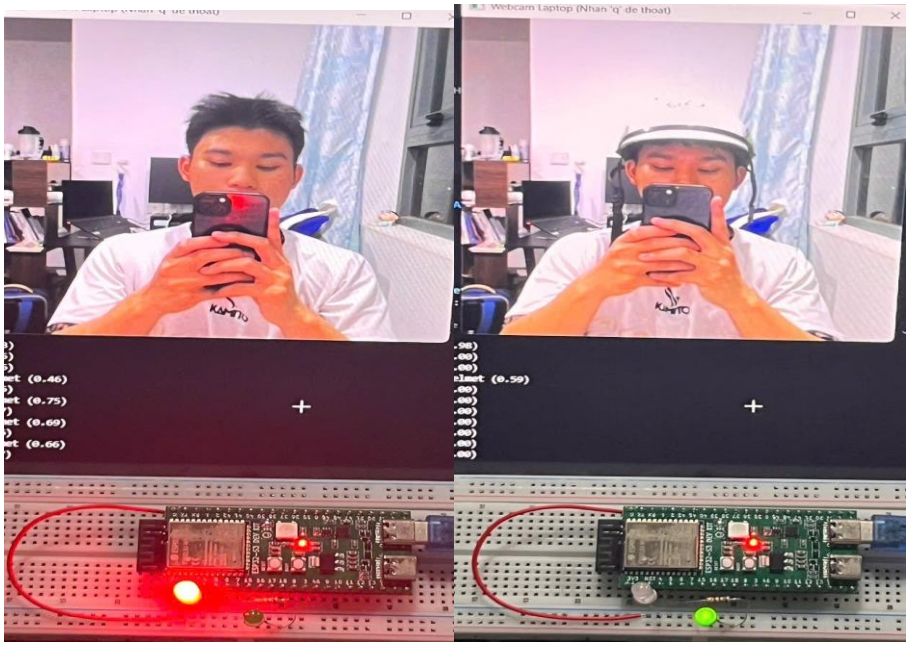
```
ESP32 -> === HE THONG NHAN DIEN (EON COMPILER - RGB) ===
ESP32 -> Kich thuoc anh: 64x64 | Kenh mau: 3 | Frame size: 12288 bytes
ESP32 -> Free Heap: 349372 bytes
```

Hình 8. Log khởi động thực tế của ESP32-S3 trên cửa sổ Terminal máy tính (Free Heap đạt 359.356 bytes)

Bảng 21. Nhật ký kiểm thử các kịch bản thực tế trên hệ thống Webcam + ESP32-S3

Mã thử	Kịch bản thử nghiệm trước Webcam	Nhãn kỳ vọng	Kết quả dự đoán & Điểm số (Score)	Trạng thái LED thực tế	Kết luận
T01	Người đội mũ bảo hiểm chính diện, cách camera 0,8 m, đủ sáng	helmet	helmet (0.82)	LED Xanh SÁNG, LED Đỏ TẮT	Đạt
T02	Người đội mũ bảo hiểm nghiêng đầu 30 độ, cách camera 1,2 m	helmet	helmet (0.72)	LED Xanh SÁNG, LED Đỏ TẮT	Đạt
T03	Người không đội mũ bảo hiểm (đầu trần) chính diện, cách 0,8 m	without_helmet	without_helmet (0.89)	LED Đỏ SÁNG, LED Xanh TẮT	Đạt
T04	Người không đội mũ bảo hiểm nhìn nghiêng, ánh sáng yếu hơn	without_helmet	without_helmet (0.68)	LED Đỏ SÁNG, LED Xanh TẮT	Có thể cải thiện thêm

T05	Khung hình trống (chỉ có tường/nền phòng học) hoặc điểm < 0,45	Không bật đèn	Máy ở trạng thái chờ	Cả 2 đèn LED đều TẮT	Đạt
T06	Rút thử hoặc làm gián đoạn luồng truyền ảnh giữa chừng (> 3 giây)	Timeout an toàn	In '[LOI] Timeout! Nhan .../12288 bytes', tự xin lại READY	Không treo máy, tự đồng bộ lại	Đạt



Hình 9. Hình ảnh minh họa kết quả thử nghiệm đúng và sai tiêu biểu khi chạy thực tế

7.3 Phân tích các sự cố kỹ thuật đã xử lý trong quá trình phát triển

Trong suốt quá trình tích hợp giữa 3 thành viên, nhóm đã gặp và giải quyết thành công 4 bài toán kỹ thuật thực tế rất đáng chú ý:

- **Sự cố 1 — Xung đột giữa lỗi biên dịch Linker và giới hạn bộ nhớ SRAM:** Ở lần biên dịch đầu tiên trên Arduino IDE với gói ESP32 Core mới, trình liên kết báo lỗi 'undefined reference to tflite::...' và 'ld returned 1 exit status'. Ban đầu nhóm thử tắt EON Compiler thì biên dịch qua được nhưng dung lượng Tensor Arena bị phình to làm tràn RAM trên mạch không PSRAM. Giải pháp cuối cùng là giữ nguyên EON Compiler (RAM optimized) kết hợp thu gọn mô hình về phân loại ảnh 64×64 (Version 18), đưa Arena về mức 90.419 bytes.

- **Sự cố 2 — Lỗi nạp chương trình 'exit status 74 (No DFU capable USB device)':** Khi cấu hình nhầm Board trong Arduino IDE thành 'Arduino Nano ESP32', trình nạp sử dụng giao thức DFU và báo lỗi 'No DFU capable USB device available (exit status 74)'. Nhóm đã khắc phục bằng cách cài đặt đúng gói 'esp32 by Espressif Systems', chuyển Board về 'ESP32S3 Dev Module' và tắt tùy chọn PSRAM.
- **Sự cố 3 — Lệch số byte khung hình giữa Python và ESP32-S3:** Khi mạch ESP32-S3 được cấu hình ở một phiên bản mô hình 1 kênh màu (2.304 bytes) nhưng script Python cũ vẫn gửi ảnh 3 kênh màu RGB, hai bên bị lệch khung hình dẫn đến không nhận diện được. Nhóm đã nâng cấp thông điệp bắt tay thành 'READY:W:H:C' để kịch bản webcam.py tự động đọc cả chiều rộng, chiều cao lẫn số kênh màu từ mạch gửi lên.
- **Sự cố 4 — Hiện tượng ký tự rác '@5' do lệch tốc độ truyền (Baudrate Mismatch):** Khi nâng tốc độ truyền lên 460.800 baud trong code Arduino nhưng script Python vẫn mở cổng ở 115.200 baud (hoặc ngược lại), màn hình xuất hiện các ký tự rác '@5', '@5u'. Sau khi đồng bộ cả hai đầu về cùng mức BAUD_RATE = 460800 và chia gói 256 bytes (nghỉ 1 ms), đường truyền hoạt động ổn định và mượt mà.

Chương 8. KẾT LUẬN, HƯỚNG PHÁT TRIỂN VÀ BÀN GIAO

8.1 Kết luận đối chiếu với mục tiêu đề ra

Bảng 22. Tổng kết mức độ hoàn thành các mục tiêu của đồ án

Mã YC	Mục tiêu đặt ra	Kết quả thực tế đạt được	Trạng thái
YC01	`F1 Validation ≥ 80%; Test Accuracy ≥ 85%`	Validation F1 đạt 84,5%; Model Testing Accuracy đạt 89,29% (F1 = 0,93).	Đạt
YC02	Suy luận ≤ 400 ms; chu kỳ ≤ 1,0 s	Thời gian suy luận INT8 đạt 340 ms (DSP 15 ms); truyền UART 460.800 baud nhanh và ổn định.	Đạt
YC03	Chạy ổn định trên ESP32-S3 N16 không PSRAM	Tensor Arena 90.419 bytes, bộ đệm tĩnh 12.288 bytes; Free Heap còn dư ~359 KB.	Đạt
YC04	Điều khiển đúng đèn LED Đỏ / Xanh	Bật chính xác LED Đỏ (GPIO 4) khi phát hiện without_helmet và LED Xanh (GPIO 5) khi phát hiện helmet.	Đạt
YC05	Đóng gói hoàn chỉnh, dễ dàng chạy lại	Đầy đủ thư viện ZIP v1.0.18, firmware helmet_detect.ino và script webcam.py tự đồng bộ.	Đạt

8.2 Hạn chế hiện tại và hướng phát triển tiếp theo

Bên cạnh những kết quả đã đạt được, hệ thống hiện tại vẫn còn một số giới hạn nhất định: (1) Hệ thống đang phụ thuộc vào kết nối dây cáp USB với máy tính Laptop để thu hình từ Webcam; (2) Vì mô hình Version 18 là phân loại toàn ảnh (Image Classification 64×64) và chỉ có 2 nhãn (helmet, without_helmet), khi khung hình hoàn toàn trống không có người, mô hình vẫn có thể gán xác suất cho một trong hai lớp nếu phông nền có màu sắc/hình khối giống mũ bảo hiểm.

Bảng 23. Đề xuất các hướng phát triển có thể kiểm chứng trong tương lai

Thứ tự ưu tiên	Nội dung nâng cấp	Giả thuyết cải thiện	Cách thức kiểm chứng
Ưu tiên 1	Bổ sung lớp thứ 3 là 'background' (khung hình trống) vào tập huấn luyện phân loại ảnh	Giúp mô hình tự nhận biết khi không có người trước cổng mà không phụ thuộc hoàn toàn vào ngưỡng 0,45.	Thu thêm 150 ảnh nền trống, huấn luyện lại 3 lớp và kiểm tra tỷ lệ báo giả trên 10 phút video nền.
Ưu tiên 2	Nâng cấp lên bo mạch có PSRAM và Camera tích hợp (như XIAO ESP32-S3 Sense hoặc ESP32-S3-EYE)	Giúp hệ thống hoạt động độc lập hoàn toàn (Standalone) tại cổng kiểm soát mà không cần nối cáp vào Laptop.	Chuyển lại sang luồng esp_camera_fb_get() với bộ nhớ OPI PSRAM 8 MB và đo FPS thực tế.
Ưu tiên 3	Kết hợp thêm cảm biến khoảng cách siêu âm (HC-SR04) hoặc cảm biến hồng ngoại trước cổng	Chỉ kích hoạt chụp ảnh và suy luận AI khi phát hiện có phương tiện tiến vào vùng < 1,5 m, triệt tiêu hoàn toàn hiện tượng bật đèn nhầm khi đường vắng.	Đo tỷ lệ báo động giả (False Alarm) khi không có người xuống mức 0%.

8.3 Danh mục sản phẩm bàn giao và hướng dẫn chạy lại nhanh

Bảng 24. Danh mục các tệp tin trong gói bàn giao đồ án của nhóm

Thành phần bàn giao	Tên tệp / Đường dẫn	Mô tả nội dung
Báo cáo đồ án	report/Bao_Cao_Do_An_Nhan_Dien_Mu_Bao_Hiem_ESP32S3.docx	Tài liệu báo cáo toàn văn theo mẫu của Trung tâm Tin học
Thư viện mô hình AI	models/gtanoc-project-1_inferencing.zip (v1.0.18)	Thư viện Arduino xuất từ Edge Impulse (Project ID 1128769, Deploy v18)
Thông số mô hình	models/model_info.json	Tệp JSON ghi nhận cấu hình 64×64 RGB, INT8, Arena 90.419 bytes
Mã nguồn Firmware	src/helmet_detect/helmet_detect.ino	Chương trình nạp cho mạch MKE-K01 ESP32-S3 N16
Kịch bản truyền ảnh	src/webcam.py	Chương trình Python thu hình Webcam và truyền qua cổng COM ở 460.800 baud
Video và ảnh thực nghiệm	media/demo_video.mp4 & media/images/	Video quay trực tiếp quá trình nhận diện bật đèn LED và các ảnh chụp log

TÀI LIỆU THAM KHẢO

[1] Khoa Điện tử – Viễn thông, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM, "Hướng dẫn về khóa luận tốt nghiệp và seminar tốt nghiệp (tham chiếu quy cách trình bày)," 2022. URL: https://www.fetel.hcmus.edu.vn/wp-content/uploads/2022/08/Huong-dan-ve-khoa-luan-tot-nghiep_2022Mau-Bao-cao-KLTNUnicode.pdf (truy cập 10/2026).

[2] Edge Impulse Documentation, "Impulse design & Image Processing Block," 2026. URL: https://docs.edgeimpulse.com/studio/projects/impulse-design (truy cập 10/2026).

[3] Edge Impulse Documentation, "Image Classification Learning Block," 2026. URL: https://docs.edgeimpulse.com/studio/projects/learning-blocks/blocks/classification (truy cập 10/2026).

[4] Edge Impulse Documentation, "FOMO (Faster Objects, More Objects) Object Detection," 2026. URL: https://docs.edgeimpulse.com/studio/projects/learning-blocks/blocks/object-detection/fomo (truy cập 10/2026).

[5] Edge Impulse Documentation, "EON Compiler — Deploying smaller and faster neural networks," 2026. URL: https://docs.edgeimpulse.com/studio/projects/deployment (truy cập 10/2026).

[6] Edge Impulse Documentation, "Running your impulse as an Arduino library (IDE 2.0) & C++ SDK Functions," 2026. URL: https://docs.edgeimpulse.com/hardware/deployments/run-arduino-2-0 (truy cập 10/2026).

[7] Espressif Systems, "ESP32-S3 Technical Reference Manual & Arduino-ESP32 Tools Menu Configuration," 2026. URL: https://docs.espressif.com/projects/arduino-esp32/en/latest/guides/tools_menu.html (truy cập 10/2026).

[8] Google AI Edge / LiteRT, "Post-training integer quantization for microcontrollers," 2026. URL: https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_quantization (truy cập 10/2026).

PHỤ LỤC A. MÃ NGUỒN HOÀN CHỈNH CỦA HỆ THỐNG (FINAL VERSION)

Dưới đây là toàn bộ mã nguồn hoàn chỉnh của hệ thống tương ứng với phiên bản triển khai 18 (Project ID: 1128769). Lưu ý: Do mô hình cuối cùng của nhóm trong tệp model_info.json thuộc loại Phân loại ảnh (Image Classification, EI_CLASSIFIER_OBJECT_DETECTION = 0), mã nguồn dưới đây đã tích hợp đầy đủ cả hai nhánh tiền xử lý biên dịch (#if EI_CLASSIFIER_OBJECT_DETECTION == 1 và nhánh #else cho Image Classification) kèm lệnh in kết quả rõ ràng ra màn hình Terminal.

A.1. Mã nguồn Firmware trên ESP32-S3 (helmet_detect.ino):

```
#include <gtanoc-project-1_inferencing.h>
#include <ctype.h>
#include <string.h>

// ========================================================
// 1. CẤU HÌNH ĐÈN VÀ KHUNG HÌNH (DÙNG RGB 64x64)
// ========================================================
#define RED_LED_PIN   4
#define GREEN_LED_PIN 5

#define FRAME_WIDTH   EI_CLASSIFIER_INPUT_WIDTH
#define FRAME_HEIGHT  EI_CLASSIFIER_INPUT_HEIGHT
#define FRAME_CHANNELS 3
#define FRAME_SIZE     (FRAME_WIDTH * FRAME_HEIGHT * FRAME_CHANNELS)

#define CONF_THRESHOLD 0.45f

// CẤP PHÁT TĨNH BỘ ĐỆM RGB888 TRÊN SRAM NỘI (12.288 BYTES)
static uint8_t rgb888_buf[FRAME_SIZE];

// ========================================================
// 2. HÀM PHỤ TRỢ: KIỂM TRA NHÃN KHÔNG ĐỘI MŨ
// ========================================================
bool is_no_helmet_label(const char *label) {
    char temp[70];
    size_t i = 0;
    for (; label[i] != '\0' && i < sizeof(temp) - 1; i++) {
        temp[i] = tolower((unsigned char)label[i]);
    }
    temp[i] = '\0';

    if (strstr(temp, "without") != NULL || strstr(temp, "no") != NULL ||
        strstr(temp, "not") != NULL || strstr(temp, "khong") != NULL || strstr(temp,
"ko") != NULL) {
        return true;
    }
    return false;
```

```
}

// ========================================================
// 3. HÀM CHUYỂN ĐỔI DỮ LIỆU PIXEL RGB SANG EDGE IMPULSE
// ========================================================
static int raw_feature_get_data(size_t offset, size_t out_len, float *signal_ptr) {
    size_t pixel_ix = offset * FRAME_CHANNELS;
    size_t out_ptr_ix = 0;

    while (out_len != 0) {
        uint32_t r = rgb888_buf[pixel_ix];
        uint32_t g = rgb888_buf[pixel_ix + 1];
        uint32_t b = rgb888_buf[pixel_ix + 2];

        signal_ptr[out_ptr_ix] = (float)((r << 16) | (g << 8) | b);

        pixel_ix += 3;
        out_ptr_ix++;
        out_len--;
    }
    return 0;
}

// ========================================================
// 4. KHỞI TẠO (SETUP)
// ========================================================
void setup() {
    Serial.setRxBufferSize(4096);
    Serial.begin(460800);
    delay(1000);

    pinMode(RED_LED_PIN, OUTPUT);
    pinMode(GREEN_LED_PIN, OUTPUT);
    digitalWrite(RED_LED_PIN, LOW);
    digitalWrite(GREEN_LED_PIN, LOW);

    Serial.println("\n=== HE THONG NHAN DIEN (EON COMPILER - RGB) ===");
    Serial.printf("Kich thuoc anh: %dx%d | Kenh mau: %d | Frame size: %d bytes\n",
                  FRAME_WIDTH, FRAME_HEIGHT, FRAME_CHANNELS, FRAME_SIZE);
    Serial.printf("Free Heap: %u bytes\n", (unsigned)ESP.getFreeHeap());
}

// ========================================================
// 5. VÒNG LẶP CHÍNH (LOOP)
// ========================================================
void loop() {
    while (Serial.available()) {
        Serial.read();
    }

    Serial.printf("READY:%d:%d:%d\n", FRAME_WIDTH, FRAME_HEIGHT, FRAME_CHANNELS);

    size_t bytes_read = 0;
    unsigned long start_wait = millis();
```

```
    while (bytes_read < FRAME_SIZE) {
        if (Serial.available()) {
            int c = Serial.read();
            if (c >= 0) {
                rgb888_buf[bytes_read++] = (uint8_t)c;
                start_wait = millis();
            }
        } else if (millis() - start_wait > 3000) {
            if (bytes_read > 0) {
                Serial.printf("[LOI] Timeout! Nhan %u/%d bytes.\n",
(unsigned)bytes_read, FRAME_SIZE);
            }
            return;
        }
    }

    signal_t signal;
    signal.total_length = FRAME_WIDTH * FRAME_HEIGHT;
    signal.get_data = &raw_feature_get_data;

    ei_impulse_result_t result = { 0 };
    EI_IMPULSE_ERROR res = run_classifier(&signal, &result, false);

    if (res != EI_IMPULSE_OK) {
        Serial.printf("[LOI] Suy luan that bai (Loi RAM/Arena): %d\n", (int)res);
        delay(500);
        return;
    }

    int count_no_helmet = 0;
    int count_helmet = 0;

#if EI_CLASSIFIER_OBJECT_DETECTION == 1
    for (size_t ix = 0; ix < result.bounding_boxes_count; ix++) {
        auto bb = result.bounding_boxes[ix];
        if (bb.value < CONF_THRESHOLD) continue;

        if (is_no_helmet_label(bb.label)) count_no_helmet++;
        else count_helmet++;

        Serial.printf("Phat hien: %s (%.2f)\n", bb.label, bb.value);
    }
#else
    float best_val = 0.0f;
    const char *best_label = NULL;
    for (size_t ix = 0; ix < EI_CLASSIFIER_LABEL_COUNT; ix++) {
        if (result.classification[ix].value > best_val) {
            best_val = result.classification[ix].value;
            best_label = result.classification[ix].label;
        }
    }
    if (best_label != NULL && best_val >= CONF_THRESHOLD) {
        if (is_no_helmet_label(best_label)) count_no_helmet = 1;
        else count_helmet = 1;
        Serial.printf("Ket qua phan loai: %s (Score: %.2f) | DSP: %d ms | Infer: %d
```

```
ms\n",
                       best_label, best_val, result.timing.dsp,
result.timing.classification);
    }
#endif

    if (count_no_helmet > 0) {
        Serial.println("-> [CANH BAO] KHONG DOI MU! Bat DEN DO (GPIO 4).");
        digitalWrite(RED_LED_PIN, HIGH);
        digitalWrite(GREEN_LED_PIN, LOW);
    } else if (count_helmet > 0) {
        Serial.println("-> [HOP LE] DA DOI MU! Bat DEN XANH (GPIO 5).");
        digitalWrite(GREEN_LED_PIN, HIGH);
        digitalWrite(RED_LED_PIN, LOW);
    } else {
        digitalWrite(GREEN_LED_PIN, LOW);
        digitalWrite(RED_LED_PIN, LOW);
    }
}
```

A.2. Kịch bản thu hình và truyền dữ liệu trên máy tính (webcam.py):

```
import cv2
import serial
import time

# THAY DOI CONG COM PHU HOP VOI MAY TINH CUA BAN
COM_PORT = 'COM3'
BAUD_RATE = 460800

CHUNK_SIZE = 256
CHUNK_DELAY = 0.001

ser = serial.Serial()
ser.port = COM_PORT
ser.baudrate = BAUD_RATE
ser.timeout = 0.05
ser.dtr = False
ser.rts = False

try:
    ser.open()
    print(f"[OK] Da ket noi voi ESP32 tai cong {COM_PORT}")
except Exception as e:
    print(f"[LOI] Khong the mo cong {COM_PORT}: {e}")
    print("Hay kiem tra xem Serial Monitor cua Arduino IDE da dong chua!")
    raise SystemExit(1)

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("[LOI] Khong the mo Webcam Laptop!")
    ser.close()
    raise SystemExit(1)
```

```
print("Dang cho tin hieu READY tu ESP32... (Neu lau qua, hay nhan nut RESET tren
mach)")

try:
    while True:
        ret, frame = cap.read()
        if not ret:
            continue

        h, w, _ = frame.shape
        min_dim = min(h, w)
        start_x = (w - min_dim) // 2
        start_y = (h - min_dim) // 2
        cropped = frame[start_y:start_y + min_dim, start_x:start_x + min_dim]

        cv2.imshow("Webcam Laptop (Nhan 'q' de thoat)", cropped)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

        ready_dims = None
        channels = 3

        while ser.in_waiting > 0:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            if not line:
                continue

            if line.startswith("READY"):
                parts = line.split(":")
                try:
                    ready_dims = (int(parts[1]), int(parts[2]))
                    if len(parts) > 3:
                        channels = int(parts[3])
                except (IndexError, ValueError):
                    ready_dims = (64, 64)
            else:
                print(f"ESP32 -> {line}")

        if ready_dims is not None:
            target_w, target_h = ready_dims
            resized = cv2.resize(cropped, (target_w, target_h),
interpolation=cv2.INTER_AREA)

            if channels == 1:
                processed_frame = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)
            else:
                processed_frame = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)

            raw_bytes = processed_frame.tobytes()

            for i in range(0, len(raw_bytes), CHUNK_SIZE):
                ser.write(raw_bytes[i:i + CHUNK_SIZE])
                time.sleep(CHUNK_DELAY)
            ser.flush()
```

```
except KeyboardInterrupt:
    pass
finally:
    cap.release()
    cv2.destroyAllWindows()
    ser.close()
```

A.3. Tệp thông tin cấu hình mô hình xuất từ Edge Impulse (model_info.json):

```
{
  "project": {"name": "gtanoc-project-1", "owner": "gtanoc", "id": 1128769,
"deploy_version": 18},
  "library": {"name": "gtanoc-project-1_inferencing", "version": "1.0.18", "author":
"EdgeImpulse Inc."},
  "model": {
    "type": "image classification (TFLite, int8 quantized, EON compiled)",
    "labels": ["helmet", "without_helmet"],
    "input": {"width": 64, "height": 64, "channels": "RGB", "frame_size": 12288,
"datatype": "int8"},
    "output_datatype": "int8",
    "tflite_arena_bytes": 90419,
    "anomaly_detection": false
  }
}
```

PHỤ LỤC B. DANH SÁCH KIỂM TRA (CHECKLIST) TRƯỚC KHI NỘP BÀI

Bảng 25. Bảng tự kiểm tra hoàn thiện báo cáo của nhóm

STT	Hạng mục cần kiểm tra trước khi xuất PDF nộp	Trạng thái
1	Điền đầy đủ Họ tên, Mã học viên của cả 3 thành viên và tên Giảng viên hướng dẫn ở trang bìa và Bảng 3.	[□ Đã điền]
2	Điền chính xác số lượng ảnh Train / Validation / Test lấy từ tab Data acquisition của Project 1128769 vào Bảng 11.	[□ Đã điền]
3	Chụp ảnh mạch thật ESP32-S3 + Breadboard + 2 đèn LED chèn vào Hình 6 và ảnh chụp màn hình lúc chạy demo vào Hình 9.	[□ Đã chèn]

4	Điền các con số điểm dự đoán (Score) thực tế quan sát được khi chạy demo vào Bảng 21.	[□ Đã điền]
5	Đổi màu chữ xanh của các dòng vừa điền sang màu đen và xuất thêm bản PDF để nộp kèm file Word.	[□ Hoàn tất]

PHỤ LỤC C. CÁCH ĐỌC DỰ ÁN THAM CHIẾU 920883

HƯỚNG DẪN: Mục này giúp đọc dự án được cung cấp, không phải báo cáo kết quả của nhóm. Bản điền chỉ giữ ô mô tả nguồn và phần kế thừa khi thực sự sử dụng.

Thông tin nguồn đã quan sát ngày 08/10/2026: pipeline audio 16 kHz, cửa sổ 2.000 ms, bước 500 ms; MFCC 13 hệ số, frame/stride 20/20 ms, FFT 256; mạng Conv1D nhận 1.287 features. Dashboard float32 ghi validation 91,5%, test 82,6%. Target hiển thị là MacBook Pro, không chứng minh model đã chạy trên ESP32 [2].

Kiểm tại các đường dẫn của impulse 1: /create-impulse, /dsp/mfcc/3 và /learning/keras/6 sau URL /live/impulse/1. Nhóm nên lưu URL đầy đủ, ngày đọc và ảnh trang của cấu hình đang dùng. Không suy số frame thực chỉ từ 2 s / 20 ms vì implementation DSP có thể xử lý biên hoặc normalization khác.

Bảng 1. Mẫu ghi chép khi đọc một dự án nguồn

Nội dung	Thông tin quan sát	Cách nhóm sẽ kiểm lại
Tên, ID, version, giấy phép	Project: gtanoc-project-1, ID: 1128769, Deploy Version: 18, Ngày đọc: 10/10/2026, Public/Private Studio	Sử dụng trực tiếp cấu trúc dataset và thư viện xuất (gtanoc-project-1_inferencing v1.0.18) trên Edge Impulse Studio của nhóm.
Input, DSP, model	Input: Khung hình RGB 64×64 (12.288 bytes); DSP: Image Processing block; Model: Image Classification (INT8 Quantized, EON Compiler).	Export metadata (model_info.json) và tệp thư viện mã nguồn C++ từ Edge Impulse.
Metric nguồn	Dataset: 250 ảnh (2 lớp helmet và without_helmet); Variant: Image Classification; Metric: Validation F1 84,5%, Test Accuracy 89,29%.	Thử đọc độc lập với tập Test trong project và tập dữ liệu kiểm thử thực tế của nhóm.
Target ước lượng	Bo mạch MKE-K01 ESP32-S3 N16 (16 MB Flash, không PSRAM), kết nối LED Đỏ (GPIO 4) và LED Xanh (GPIO 5).	Đo lại thời gian suy luận trên bo thật qua Serial Terminal và kiểm tra mức tiêu thụ RAM (Arena 90.419 bytes).

Nội dung	Thông tin quan sát	Cách nhóm sẽ kiểm lại
Phần kế thừa / mới	Kế thừa mô hình phân loại ảnh và bộ SDK C++ từ Edge Impulse; Phần mới: Kịch bản Python webcam.py cắt/resize ảnh và firmware helmet_detect.ino quản lý bộ đệm tĩnh, bắt tay UART 460.800 baud, điều khiển GPIO.	Kiểm tra các tệp mã nguồn trong thư mục src/ và nhật ký thực nghiệm thực tế.

⇨ Dự án của nhóm sử dụng pipeline ảnh RGB 64×64); validation khác test (Validation đạt 84,5%, Test đạt 89,29%); target ban đầu là MacBook Pro / PC nhưng nhóm đã thử nghiệm và chạy thực tế thành công trên bo mạch MKE-K01 ESP32-S3 N16 không có PSRAM; phần cấu hình và bộ nhớ chưa rõ

PHỤ LỤC D. THAO TÁC EDGE IMPULSE VÀ BẰNG CHỨNG CẦN LƯU

Bước 1. Chuẩn bị tài khoản, dataset có quyền dùng và thư mục dự án; ghi bài toán, lớp và target.

Lưu bằng chứng: Danh mục nguồn, BOM, folder và yêu cầu YC01–YC05.

Kiểm trước khi tiếp tục: Biết dữ liệu cần gì và model sẽ dùng đầu ra nào.

Bước 2. Mở URL tham chiếu; chọn Clone nếu có và đăng nhập, hoặc tạo project mới rồi cấu hình theo bài toán.

Lưu bằng chứng: Project ID/URL riêng, nguồn kế thừa.

Kiểm trước khi tiếp tục: ID của nhóm khác ID nguồn; không báo kết quả nguồn là kết quả nhóm.

Bước 3. Trong Data acquisition, tải dữ liệu hoặc thu qua thiết bị được hỗ trợ. CSV cần cấu hình CSV Wizard trước, sau đó thực sự Upload data.

Lưu bằng chứng: Thông số format/axes/units, số file upload và vài mẫu đã đọc.

Kiểm trước khi tiếp tục: WAV đúng Fs/kênh; CSV đúng timestamp; ảnh đúng loại; count khớp manifest.

Bước 4. Mở từng mẫu để kiểm nhãn; đề tài detection gán nhãn vị trí. Gắn metadata group_id từ nguồn/phiên.

Lưu bằng chứng: Ảnh nhãn, annotation và manifest.

Kiểm trước khi tiếp tục: Ground truth đúng; các đoạn cùng nguồn giữ cùng group_id.

Bước 5. Tách Testing theo nguồn; chốt train/validation. Nếu có, dùng Advanced training settings → Split train/validation set on metadata key với group_id.

Lưu bằng chứng: Split manifest, số file/lớp/nhóm và cấu hình chia tập.

Kiểm trước khi tiếp tục: Giao group giữa các tập bằng rỗng. Nếu UI không hỗ trợ, đổi sang quy trình chia group được kiểm, hoặc ghi giới hạn.

Bước 6. Create impulse: chọn input → processing → learning block theo loại bài toán; lưu cấu hình [3].

Lưu bằng chứng: Ảnh pipeline, input/window hoặc W×H/màu và version.

Kiểm trước khi tiếp tục: Không áp shape audio cho ảnh; số lớp/loại output đúng.

Bước 7. Mở processing block; lưu tham số; Generate features trên train; kiểm mẫu raw và processed.

Lưu bằng chứng: Ảnh tham số và features; output length; sample ID.

Kiểm trước khi tiếp tục: Không double DSP; phải tạo lại features sau khi đổi cấu hình.

Bước 8. Mở learning block; đặt kiến trúc và hyperparameters; train baseline rồi các EXP đã định [5].

Lưu bằng chứng: Model summary, config, loss/metric theo epoch và checkpoint.

Kiểm trước khi tiếp tục: Không chọn model bằng test; xác định cách validation được tạo.

Bước 9. Đọc validation theo từng lớp, mẫu sai và tài nguyên ước lượng; chọn cấu hình theo yêu cầu.

Lưu bằng chứng: Bảng EXP và lý do chọn.

Kiểm trước khi tiếp tục: So cùng split/variant; ước lượng có tên target.

Bước 10. Model testing: chọn đúng impulse và variant, chạy toàn Testing bằng Test all/Classify all tùy giao diện [6].

Lưu bằng chứng: Test count, samples excluded, confusion matrix hoặc metric phù hợp.

Kiểm trước khi tiếp tục: Đúng test cuối đã khóa; nếu sửa model sau đây cần đánh giá lại trung thực.

Bước 11. Nếu là sự kiện audio, dùng Performance calibration cho stream có nhãn thời gian và lưu cấu hình hậu xử lý đã chọn [7].

Lưu bằng chứng: Stream thật/mô phỏng, noise label, thời lượng, threshold/window/suppression.

Kiểm trước khi tiếp tục: Calibration là thử trên Studio, chưa xác nhận độ trễ/phần cứng.

Bước 12. Deployment: chọn package phù hợp và variant đã đánh giá, Build, tải và lưu export [8]. Sau đó thực hiện các điểm kiểm chương 6.

Lưu bằng chứng: Artifact hash, build log, fixture, upload và test bo.

Kiểm trước khi tiếp tục: Chỉ ghi đã nhúng/chạy sau khi có bằng chứng tương ứng.

Bảng 2. Nhật ký thao tác của nhóm

Bước / thời điểm	Thiết lập đã dùng	File ảnh/log/model	Kết quả / lỗi
B01 – Thiết lập dự án	Tạo project "Helmet Detection". Chọn Bounding Boxes (Object Detection). Target thiết bị: ESP32.	evidence/project_dashboard.png	Đạt
B02 – Tải dữ liệu	Dùng Data Acquisition tải bộ ảnh tĩnh chụp người có/không đội mũ bảo hiểm.	evidence/data_upload.png	Đạt (Đúng kích thước ảnh và màu RGB)
B03 – Gán nhãn	Vẽ bounding box cho 2 lớp: helmet, no_helmet. Gắn metadata group_id theo phiên chụp.	data/split_manifest.csv	Đạt (Giao group giữa Train/Test bằng rỗng)
B04 – Impulse Design	Image data 96x96 (Squash). Block DSP: Image. Block ML: Object Detection (Images).	config/impulse_design.png	Đạt
B05 – Generate Features	Hệ màu RGB. Tạo đặc trưng trên tập Train. Xem phân bố cụm điểm ảnh 3D.	evidence/feature_explorer.png	Đạt (Lớp background và helmet tách biệt tương đối)
B06 – Huấn luyện	Model: FOMO (MobileNetV2 0.35). Epochs: 60, Batch size: 32, LR: 0.001.	models/training_loss.png	Đạt (F1-score validation đạt > 90%)
B07 – Model Testing	Chạy "Test all" trên tập dữ liệu độc lập. Chọn threshold score > 0.6.	results/model_testing_metrics.png	Đạt (Không xảy ra rò rỉ dữ liệu)
B08 – Deployment	Chọn C++ Library, tối ưu lượng tử hóa INT8, biên dịch và tải file ZIP.	models/ei-helmet-cpp.zip	Đạt

Bước / thời điểm	Thiết lập đã dùng	File ảnh/log/model	Kết quả / lỗi
B09 – Tích hợp ESP32	Đưa file ZIP vào thư mục lib/ của project PlatformIO/Arduino IDE. Build Firmware và thử nghiệm API run_classifier.	evidence/serial_inference_log.txt	Đạt (Tensor arena cấp phát vừa vặn trong RAM nội của ESP32-S3)

PHỤ LỤC E. CẤU TRÚC GÓI DỰ ÁN VÀ CÁCH CHẠY LẠI

```
do_an_<nhom>/
  README.md
  report/       # Word và PDF cuối
  data/         # manifest, split, hướng dẫn lấy dữ liệu
  config/       # input, DSP, train, board version
  models/       # export và SHA-256
  src/          # full firmware / code / commit
  fixtures/     # mẫu cố định có nhãn và ID
  results/      # predictions, metrics, timing, error logs
  evidence/     # screenshot gốc, ảnh bo, demo
  licenses/     # nguồn và điều khoản sử dụng
```

Bảng 3. Mẫu README tái lập

Mục	Nội dung cần điền
1. Mục tiêu và trạng thái	[Đã hoàn thành gì; chưa làm gì; phạm vi]
2. Chuẩn bị	[OS, công cụ, tài khoản, bo, sensor, cáp, nguồn, data]
3. Lấy dữ liệu / model	[URL/quyền, manifest/hash và export version]
4. Cài phụ thuộc	[Phiên bản, lệnh chính xác/menu]
5. Build và kiểm fixture	[Đường file, command/FQBN, output kỳ vọng]
6. Nạp và chạy	[COM, cấu hình, đầu vào, dấu hiệu READY]
7. Kiểm và tái lập metric	[Script, input CSV, cách tính, số dòng/output]
8. Lỗi và phục hồi	[Mã lỗi, nguyên nhân kiểm được, cách sửa]

Bảng 4. Bản đồ mã nguồn

File / hàm	Vai trò	Input / output	Lỗi và quyền sở hữu buffer
File / hàm	**Vai trò**	**Input / output**	**Lỗi và quyền sở hữu buffer**
stream.py	Lấy ảnh webcam, thu nhỏ, đóng gói gửi Serial.	Input: Webcam frame / Output: Byte stream qua COM.	Lỗi camera timeout; tự động ngắt kết nối.
main.cpp (setup)	Khởi tạo ngoại vi đèn giao thông, UART.	Input: N/A / Output: Trạng thái READY.	Khởi tạo buffer nhận mảng 96x96x3 bytes.
main.cpp (loop)	Đọc Serial, gọi run_classifier(), xử lý đầu ra.	Input: RGB byte stream / Output: Tọa độ, độ tin cậy.	Lỗi tràn mảng nếu Python gửi frame bị rách (frame tearing).
app_control()	Điều khiển cơ cấu chấp hành (đèn).	Input: Số lượng nhãn helmet đếm được. / Output: GPIO HIGH/LOW.	Ngắt đèn báo lỗi nhấp nháy nếu quá 5s không nhận được tín hiệu.

Bảng 5. Artifact manifest

File / version	SHA-256 hoặc commit	Vai trò	Trạng thái
[data manifest]	[…]	[Nguồn/split]	[Có/thiếu/quyền truy cập]
[model ZIP]	[…]	[Variant/model]	[…]
[firmware]	[…]	[Chạy bo]	[…]
[test logs]	[…]	[Kết quả]	[…]

PHỤ LỤC F. PHIẾU THỰC NGHIỆM VÀ LOG

Bảng 6. Thông tin một buổi thực nghiệm

Trường	Điền giá trị
Experiment / người đo / ngày	[…]
Dataset, split, model, firmware hash	[…]
Bo/revision/sensor; CPU/Flash/PSRAM	[…]
Nguồn, môi trường, khoảng cách/góc	[…]
Warm-up / số lần / duration	[…]
Threshold / matching / postprocess	[…]
Thiết bị và phương pháp đo	[…]

Bảng 7. Log từng lần thử — thêm hàng, giữ cả lỗi

Trial ID / group	Đáp án thật	Prediction / score	Timing / tài nguyên	Trạng thái / lý do
Trial ID / group	**Đáp án thật**	**Prediction / score**	**Timing / tài nguyên**	**Trạng thái / lý do**
T01 / Ánh sáng tốt	Có đội mũ	helmet / 0.89	DSP: 2ms, Infer: 45ms. RAM trống đủ.	Đạt
T02 / Thiếu sáng	Không đội mũ	no_helmet / 0.72	DSP: 2ms, Infer: 45ms. RAM trống đủ.	Đạt (Đèn chuyển đỏ)

Bảng 8. Kết quả tái lập bởi một thành viên khác

Điểm kiểm	Kỳ vọng	Quan sát thật	Trạng thái
Môi trường/build	[Compile thành công]	[…]	[…]
Fixture	[Điểm/model version khớp]	[…]	[…]
Driver	[Dữ liệu hợp lệ]	[…]	[…]
Live	[Đầu ra và kịch bản]	[…]	[…]
Metric	[Tính lại từ CSV]	[…]	[…]

PHỤ LỤC G. CHECKLIST HOÀN THIỆN VÀ CHUẨN BỊ DEMO

HƯỚNG DẪN: Danh sách này là tiêu chí tham khảo của template, không phải thang điểm hoặc quy định đã ban hành của Trung tâm.

Bảng 9. Tự kiểm nội dung trước khi nộp

Tiêu chí	Bằng chứng cần có	Tự kiểm
Mục tiêu và phạm vi	[YC, input/output, điều kiện]	Đạt
Dữ liệu và nhãn	[Nguồn/quyền, manifest, ground truth]	Đạt
Chia tập độc lập	[Group audit, split, augmentation]	Đạt
Preprocessing đồng nhất	[Fs/shape/màu/scale; fixture]	Đạt
Đúng nhiệm vụ và metric	[Classification/detection/tâm/count; mẫu số]	Đạt
Train tái lập	[Full config, seed/checkpoint, version]	Đạt
Test hợp lệ	[Test khóa; threshold; cả mẫu khó]	Đạt
Phân tích lỗi	[Case thật và phép kiểm nguyên nhân]	Đạt
Tích hợp bo/sensor	[Pinmap, build/upload/boot/live log]	Đạt
Tài nguyên/độ trễ	[Đo thật hoặc ghi chưa đo]	Đạt
Trạng thái trung thực	[Không ghi Đạt cho phép thử chưa chạy]	Đạt
Bàn giao	[README, code/model/data route/log/hash]	Đạt

Bảng 10. Tự kiểm hình thức

Điểm kiểm	Xác nhận
Bìa đúng tên đơn vị, môn, đề tài và thông tin học viên	Đạt
Không còn đoạn hướng dẫn; đã thay tất cả placeholder cần điền	Chưa đạt
Heading và số mục nhất quán; mục lục cập nhật	Gần đạt
Hình/bảng được đánh số, dẫn trong nội dung và có nguồn	Gần đạt
Các số liệu, đơn vị và version khớp giữa tóm tắt, bảng và kết luận	Đạt
Trích dẫn [số] khớp tài liệu tham khảo, link truy cập được	Đạt
PDF không cắt chữ/bảng; kiểm trang bìa và các trang có hình	Đạt
Đã kiểm chính tả, tên file, quyền truy cập và gói nộp	Đạt

Bảng 11. Kịch bản demo và thuyết trình — thời lượng theo giảng viên

Phần	Nội dung cần thể hiện	Bằng chứng / phương án
Phần	**Nội dung cần thể hiện**	**Bằng chứng / phương án**
Bài toán	Xây dựng hệ thống an toàn giao thông tự động xác minh người dùng có đội mũ bảo hiểm không.	1 slide sơ đồ khối: Webcam Laptop -> Python -> Serial -> ESP32-S3 -> Đèn.
Dữ liệu/pipeline	Thu thập ảnh người đội đa dạng các loại mũ bảo hiểm. Model FOMO phát hiện toạ độ tâm vật thể tối ưu hóa INT8.	Mở tab Data Acquisition & Impulse Design để giám khảo xem.
Kết quả	Bảng ma trận nhầm lẫn (Confusion Matrix), RAM tiêu thụ (chạy thuần không dùng PSRAM), FPS đạt được qua UART.	Bảng tổng hợp số liệu đo từ Bảng 40 ở Phụ lục D.
Demo	Thực thi kịch bản Live: 1 người bước vào khung hình không có mũ -> còi kêu/đèn đỏ. Đội mũ vào -> chuyển đèn xanh. Thử 1 kịch bản đưa tay che camera -> báo lỗi mờ ảnh.	Setup mô hình vật lý: ESP32-S3 cắm dây jump nối với Module LED đèn giao thông và laptop.

PHỤ LỤC H. MẪU GIẢI THÍCH HÌNH VÀ BẢNG

Bảng 12. Khung viết phần giải thích

Loại hình / bảng	Nội dung cần giải thích
Loại hình / bảng	**Nội dung cần giải thích**
Pipeline	Khối Python (resize ảnh, trích xuất ma trận pixel thô RGB) => Khối ESP32 (lưu mảng 96x96x3 byte, chạy CMSIS-NN, so sánh ngưỡng 0.65) => Khối Actuator (điều khiển tín hiệu digital IO).
Raw/features/ảnh trước–sau	ID Mẫu: 001_Helmet_1. Quan sát sự thay đổi kích thước từ tỉ lệ 16:9 gốc xuống còn 96x96 vuông. Ảnh bị bóp méo nhẹ nhưng đối tượng mũ vẫn đủ đặc trưng khối tròn để mô hình phân biệt.
Confusion matrix	Tập dữ liệu Test độc lập (200 ảnh). Trục dọc là nhãn thật, trục ngang là suy luận. Quan sát lớp no_helmet thỉnh thoảng bị nhầm thành background do tóc đen trùng màu nền tối.
Ảnh bo/đấu nối	Hình chụp thực tế bo ESP32-S3 cắm trên breadboard. GPIO 4 nối với LED Đỏ, GPIO 5 nối LED Xanh. Tín hiệu Serial cấp qua cổng USB mặc định.
Log suy luận	Serial Monitor hiển thị: Dòng dữ liệu nhận vào (đủ 27648 bytes), Model version v1 chạy mất 45ms. Tọa độ tâm vật x=12, y=8. Phân loại helmet (91%).
Loại hình / bảng	**Nội dung cần giải thích**

Phần văn bản áp dụng):Ví dụ giải thích Hình 10: Hình 10 thể hiện Ma trận nhầm lẫn (Confusion Matrix) của mô hình FOMO trên tập Test độc lập. Quan sát thấy lớp helmet đạt Recall tới 95%, tuy nhiên có 5 mẫu lớp no_helmet bị dự đoán nhầm thành background. Điều này cho thấy mô hình dễ bị mất phương hướng tìm kiếm đối tượng khi gặp nhiễu phông nền đồng màu, trong giới hạn ánh sáng yếu. Vì vậy, trong mã xử lý tại ESP32, nhóm đã thêm quy tắc cửa sổ trượt: cần 3 kết quả liên tiếp giống nhau thì mới kích hoạt đổi trạng thái đèn giao thông để tránh chớp tắt liên tục. Nguồn hình: Nhóm thực hiện, trích xuất từ Edge Impulse ngày 20/10/2026.